Optimised Approximate Range Filters

Andrei Ogurtsov
Supervised by Artem Anisimov

May 20, 2026

Abstract

Range filters are small in-memory structures attached to every SSTable of a log-structured merge-tree (LSM-tree); they answer "is the interval $[a, b]$ disjoint from the stored key set?" with one-sided error, allowing the storage engine to skip a disk fetch when the answer is yes. **?** prove that any such filter needs $\log_2(\mathcal{L}/\varepsilon)$ bits per key in the worst case, and give a matching two-layer construction — a locality-preserving hash followed by an Exact Range Emptiness (ERE) backend — that the authors describe as "mainly theoretic in nature and thus very complicated to implement". Existing practical filters either match this bound under any workload (Rosetta, Grafite, Memento) or beat it by assuming a specific data shape and losing the worst-case guarantee (SuRF, SNARF). None exploits the cluster-and-gap structure typical of production LSM keys.

This thesis re-engineers the Goswami construction and adds a distribution-aware layer on top. We implement Goswami's Weak Prefix Search index in full and show empirically that adaptive binary/linear search on packed suffixes is 13–30× faster at the bucket sizes that arise at per-SSTable scale, which lets us collapse the two ERE metadata vectors into one, saving 24% of the metadata footprint. For sparse inputs we replace the locality-preserving hash with bit truncation, removing the $\log_2 \mathcal{L}$ term from the per-key budget. On top of this backend we add Segmented Approximate Range Emptiness (Seg-ARE), a single-pass density-based segmentation that routes dense clusters to exact-mode sub-filters and sparse tails to the truncation hash.

On the Search on Sorted Data (SOSD) benchmark at False Positive Rate $\leq 10^{-3}$, the resulting filter reaches 3.7 bits per key on Wikipedia timestamps, 7.7–9.4 on synthetic clustered keys, and matches the strongest baselines on uniform and Facebook user IDs, while preserving the worst-case Goswami guarantee on adversarial inputs.

Contents

1 Introduction **2**

1.1 Motivation . . . 2

1.2 Existing Range Filters . . . 3

1.3 Our Contributions . . . 5

1.4 Notation . . . 7

1.5 Problem Statement . . . 8

1.5.1 Information-Theoretic Bounds . . . 8

1.5.2 The Role of the Hash Function . . . 9

2 Exact Range Emptiness **10**

2.1 Succinct Representation (SODA 2015, §3.2) . . . 10

2.1.1 High-Level Structure . . . 10

2.1.2 Original ERE Structure . . . 10

2.1.3 Local Search in Buckets . . . 11

2.1.4 One-Vector Optimization . . . 11

2.1.5 Block sizes and Lookup strategy . . . 13

2.1.6 Weak Prefix Search . . . 15

2.1.7 Improvements to the Grafite Backend . . . 16

2.1.8 Space Analysis . . . 17

2.1.9 Complexity Summary . . . 17

3 Key Distributions **18**

3.1 Key Distributions in Practice . . . 18

3.1.1 Common Industrial Distributions . . . 18

3.1.2 SOSD Benchmark Datasets . . . 18

3.1.3 Synthetic Distributions . . . 20

3.1.4 An assumption on queries . . . 20

3.2 Query generation . . . 21

3.2.1 Empty queries for False Positive Rate Measurement . . . 21

3.2.2 Combined queries (for latency) . . . 21

4 Hash Function Approaches **23**

4.1 Pairwise-Independent Hash (SODA Baseline) . . . 23

4.1.1 The Hash Function . . . 23

4.1.2 ARE Construction With SODA Hash . . . 24

4.1.3 Empirical Validation . . . 24

4.2 Prefix Truncation . . . 25

4.2.1 The Hash Function . . . 25

4.2.2 ARE Construction With Truncation . . . 26

4.2.3 The Price of Contiguity . . . 27

4.3 Adaptive Filter . . . 29

4.3.1 Exact Mode . . . 29

4.3.2 When Does Exact Mode Trigger? . . . 30

4.4 Grafite: A Closer Look . . . 30

4.4.1 Parameter Choices and Their Consequences . . . 30

4.4.2 Lossless Elias–Fano Fallback . . . 31

5 Segmented ARE Filters **32**

5.1 1D DBSCAN in $O(n)$ . . . 33

5.2 Seg-ARE: Design and Algorithm . . . 33

5.2.1 Algorithm . . . 34

5.2.2 Fallback . . . 34

5.2.3 Query . . . 34

5.2.4 FPR Guarantee . . . 35

5.3 Seg-ARE vs Grafite . . . 35

6 Experimental Evaluation **38**

6.1 Benchmark Setup . . . 38

6.2 Workload Parameter Choices . . . 39

6.3 ERE Backend Comparison: Two-Vector vs One-Vector . . . 41

6.4 ERE Bucket Occupancy on SOSD . . . 42

6.5 Comparison Tables . . . 43

6.5.1 Bits per Key to Reach FPR $\leq 10^{-3}$ . . . 44

6.5.2 Query Latency at Operating Point . . . 44

6.5.3 Build Throughput at Operating Point . . . 45

7 Conclusion **47**

7.1 Summary of Contributions 47

7.2 When Seg-ARE Wins, Ties, and Loses 47

7.3 Limitations 48

7.4 Future Work 48

Chapter 1

Introduction

1.1 Motivation

Modern key–value stores — RocksDB, LevelDB, Cassandra, HBase — are built on the *log-structured merge-tree* (LSM-tree) [**?**]. The LSM-tree is a write-optimised data structure: incoming writes accumulate in an in-memory buffer and flush to disk as immutable Sorted String Tables (SSTables) at the top level, while a background compaction process merges and promotes files to successively larger levels. The structure converts random writes into sequential I/O at the cost of *read amplification*: a single lookup has to consult an SSTable at every level.

To avoid reading every SSTable during a lookup, every file carries a small in-memory *filter* in its metadata. The filter answers a membership question: its answer may be a false positive with probability $\leq \varepsilon$, while false negatives are forbidden. If the filter indicates that a key is not present in an SSTable, the engine skips the file with zero disk I/O. The Bloom filter [**?**] fills this role for point lookups and is the default in production LSM-trees [**??**]. Answering a range query with a Bloom filter, however, requires probing every key in the interval individually — $O(\mathcal{L})$ probes in the worst case, which is impractical at LSM scale [**?**]. Range-shaped workloads — short scans, prefix lookups over hierarchical keys (file paths, S2 cells, URLs), and time-window queries — are common in production LSM workloads [**?**].

For range queries we need a *range filter*: given a query interval $[a, b]$ of length at most $\mathcal{L}$, decide whether $[a, b] \cap S = \emptyset$ under the same false-positive guarantee. **?** prove a tight worst-case lower bound of $\log_2(\mathcal{L}/\varepsilon)$ bits per key and give a matching construction — a locality-preserving hash followed by an *Exact Range Emptiness* (ERE) backend — but their construction is, in their successors' own words, "mainly theoretic in nature and thus very complicated to implement" [**?**].

The bound is worst-case, however, and **?** make the escape clause explicit: it holds only *without further assumptions on the data or workload*, so a filter tuned to a specific data or workload can do better. Existing practical filters respond to this in two ways: either go below the Goswami bound by assuming a specific data distribution, losing the worst-case FPR guarantee (SuRF, SNARF), or stay at the bound and guarantee the FPR under any workload (Rosetta, Grafite, Memento) [**???**]. Many production LSM keys, however, have exploitable structure: they pack densely across the used range, spread across the universe, or concentrate in dense regions separated by large gaps [**?**]. None of the filters above uses this.

This thesis decomposes, simplifies, and accelerates the Goswami-style *Exact Range Emptiness* (ERE) backend. We implemented Goswami's internal Weak Prefix Search (WPS) index [**?**] in full — the part Grafite calls "mainly theoretic in nature and thus very complicated to implement" [**?**] — and measured its cost empirically. The measurements show that adaptive

binary/linear search on packed suffixes outperforms WPS at the bucket sizes that arise at per-SSTable scale on both axes — 59 bits per key of auxiliary metadata removed and per-query latency reduced by 13–30× relative to WPS. Once WPS is gone, Goswami's two metadata vectors D_1, D_2 collapse into a single Elias–Fano-encoded vector D — a further 24% metadata saving and the same backend Grafite arrives at by implementation-simplicity arguments (Chapter 2).

On top of this backend we add a distribution-aware segmentation that detects dense clusters and handles them exactly — going below Goswami's bound on real-world data (Search on Sorted Data benchmark [**?**]) while preserving the guarantee on adversarial inputs.

1.2 Existing Range Filters

We survey the five practical range filters most directly comparable to our work. We describe each by the technique that determines its space–accuracy trade-off and contrast it with the Goswami lower bound (Section 1.5.1).

Trie-based: SuRF. The Succinct Range Filter [**?**] stores the set of keys as a Fast Succinct Trie — a compact bitmap-based succinct trie encoding at about 10 bits per node [**?**, §3.1]. A range query descends the trie to the boundary of the query interval. The cost of the query is set by the trie depth, which reflects the key length in the universe (e.g. ≤ 8 levels for 64-bit integers under a byte-wide fanout), but *not* the query length $\mathcal{L}$; and the FPR has no explicit dependence on $\mathcal{L}$ either — unlike a linear-probe Bloom filter, which would give FPR $\approx \mathcal{L}\varepsilon$ for a range of length $\mathcal{L}$. This makes SuRF suitable for queries of almost any length without a per-query penalty.

The price of this flexibility is the absence of a worst-case FPR guarantee. A false positive occurs when the prefix of a query key coincides with a stored key prefix, so the FPR is set by the trie depth and by the *correlation between queries and data*. The authors themselves acknowledge that "the point query FPR of SuRF-Base [can be] 100%" in the adversarial case [**?**, §3.2]; in practice queries and keys are "almost always correlated to the stored keys" [**?**, §3.1], and the empirical FPR on the Yahoo! Cloud Serving Benchmark (YCSB) workload is 4% on 64-bit integer keys and 25% on email addresses [**?**, §4.2]. The space is not parameterized by ε either: SuRF-Base costs about 10 bits per key for 64-bit integers and 14 bits per key for emails regardless of the target FPR.

The headline RocksDB result — up to 5× throughput on closed-seek queries with SuRF-Real over the Bloom-filter configuration, on a synthetic 100 GB time-series workload with 128-bit timestamp+sensor keys, all filters sized to 14 bits per key [**?**, §6, Fig. 13] — is an *end-to-end* effect of skipped SSTable I/O, not a faster filter operation. A Bloom filter cannot prune range queries at all ("the Bloom filter does not help range queries and is equivalent to having no filter" [**?**, §6]), so without a range filter RocksDB fetches candidate blocks from every level; SuRF eliminates this I/O. The 5× holds when 99% of queries return empty; at 10% empty the gap narrows to roughly 1.5× (read from Fig. 13).

Bloom cascade: Rosetta. Rosetta [**?**] indexes every binary prefix of every key in a Bloom cascade — one filter per prefix length, $\log_2 \mathcal{L} + 1$ filters in total when a query-length bound $\mathcal{L}$ is known and as many as the key length otherwise [**?**, §2.1, §3.1]. A range query of length $\mathcal{L}$ decomposes into at most $2\log_2 \mathcal{L}$ power-of-two-aligned sub-intervals; for each one, Rosetta

probes the Bloom filter at the root of the corresponding sub-tree and, on a positive answer, recursively descends into deeper levels [?, §2.2.1]. Because Bloom filters are insensitive to the key distribution, the FPR guarantee holds under any workload. Under the "first-cut" bit-allocation policy [?, §2.3], the hierarchy occupies $1.44 \cdot n \log_2(\mathcal{L}/\varepsilon)$ bits — the Goswami lower bound multiplied by the standard $1/\ln 2$ Bloom-filter overhead [?, §3.1]; the deployed system redistributes the per-level budget using runtime query statistics gathered between LSM compactions [?, §2.4]. The authors are explicit about the cost: "due to the multiple probes required (for every prefix) Rosetta has a high probe cost" [?, §2]. On a 50M uint64-key benchmark at 22 bits per key, Rosetta is up to 4× faster end-to-end than SuRF and up to 40× faster than default RocksDB on short and medium range queries [?, §5, Fig. 5(A1), (D)]; the advantage decreases on long ranges.

Learned: SNARF. SNARF [?] fits a piecewise-linear monotonic model F to the empirical Cumulative Distribution Function (CDF) of the key set and uses it to map each stored key x to bit position $\lfloor F(x) \cdot nK \rfloor$ in a sparse bit array of size nK. The set bit positions are then stored either with Golomb coding [?] (default, close to the information-theoretic optimum) or with Elias–Fano coding (faster queries at slightly higher space cost). The compressed array is split into fixed-size segments so that a query only decompresses the segments overlapping its interval [?, §2.3.2]. The parameter K controls the bit array size ($|B| = nK$); total space is $\approx 2.4 + \log_2 K$ bits per key [?, §3].

On already-sorted input SNARF builds in $O(n)$: sampling, fitting linear segments between adjacent sample points, filling the bit array and compressing it are all linear. A range query costs $O(\log n + S)$, where $\log n$ is the binary search in the model's sample array and S is the number of decompressed segments.

SNARF's FPR analysis rests on two assumptions: the model maps keys approximately uniformly into the bit array, and queries are uncorrelated with stored keys. Under these assumptions FPR $\approx 1/K$ for point queries and range queries, with no explicit dependence on the range length [?, §3]. Setting $K \approx 1/\varepsilon$ gives total space $\approx 2.4 + \log_2(1/\varepsilon)$ bits per key, which is lower than Goswami's bound by $\log_2 \mathcal{L}$ bits per key: SNARF goes below Goswami's bound when its assumptions about the data hold. On the Search on Sorted Data (SOSD) benchmark [?] SNARF reports up to 50× better FPR than SuRF and 100× better FPR than Rosetta at matched memory [?, §2].

Both assumptions break in practice. The paper itself warns about correlated queries: "when a query end point is consistently close to an actual key but the query does not include a key, it may yield a false positive in SNARF and SuRF" [?, §4.1], and recommends Rosetta for short, highly correlated range queries. The uniform mapping assumption fails empirically on real-world distributions with structured sparsity. On the OpenStreetMap (OSM) dataset from SOSD — cell IDs spanning nearly the full 64-bit universe yet concentrated in a small fraction of it — the paper reports that SNARF reaches FPR $\leq 10^{-4}$ at < 10 bits per key only "for most cases", and Fig. 5(B) shows the OSM range-query curve flooring well above the 10^{-4} level reached on the other SOSD datasets, even at 16 bits per key [?, §6.1.2, Fig. 5(B)].

Practical realisation of Goswami: Grafite. Grafite [?] is the closest related work to this thesis. The authors explicitly frame it as a simplified, practical instantiation of the Goswami construction [?, §3]: $h(x) = (q(\lfloor x/r \rfloor) + x) \bmod r$ with pairwise-independent q maps the universe U down to a smaller range $[0, r)$, with $r = n\mathcal{L}/\varepsilon$; the resulting hash codes are stored using Elias–Fano coding, and a range query reduces to a single predecessor probe [?, Algorithm 2]. Theorem 3.4 establishes a space bound of $n \log_2(\mathcal{L}/\varepsilon) + 2n + o(n)$ bits and a query time of $O(\log \mathcal{L}/\varepsilon)$ —

matching the Goswami lower bound to within $O(n)$ bits. Construction takes $O(n \log n)$ time, dominated by sorting the hash codes prior to the Elias–Fano encoding [?, Algorithm 1].

The authors' own framing of the headline result reads: "Grafite is the only design to date to achieve a robust and predictable false positive rate across all combinations of datasets, query workloads, and range sizes, while also providing faster queries and construction times, and dominating all competitors in the case of correlated query workloads" [?, §1].

Because Grafite is the main baseline of this thesis, we devote a separate detailed analysis to it in Section 4.4 — internals, parameter choices, and improvements we identified during the empirical comparison.

Dynamic extension: Memento. Memento [?] adds online insertions and deletions that preserve the FPR guarantee for any workload [?, §1, §4]. It is built on top of a Rank-and-Select Quotient Filter: each *keepsake box* — a group of keys with a common fingerprint within one run — stores that fingerprint together with a contiguous list of fixed-length key suffixes ("mementos"), all packed via Robin Hood hashing [?, §3, §4].

On static workloads Memento has the same memory usage as Grafite but 1.5× higher FPR [?, §7.1, Fig. 9]. The small gap reflects Memento's $\sim$3.3n-bit metadata overhead versus Grafite's $2n$ [?, §6]. Range queries take $O(\log_2 \mathcal{L})$ CPU operations and 1–2 random memory accesses, both holding against any workload [?, §6, Table 1]. This matches Grafite on the cache-miss bound; in practice Memento is slightly slower than Grafite [?, §7.1].

The authors explicitly note that static range filters suffice for the immutable files of an LSM-Tree deployment [?, §1]; Memento targets workloads where the key set itself is mutable — single-global-filter LSM-trees, B-Tree indexes, and systems that mix transactional and analytical workloads. Because our thesis stays within the static setting, Memento is orthogonal rather than a head-to-head competitor; on the static axis it lands close to Grafite.

1.3 Our Contributions

Positioning. Rosetta, Grafite, and Memento all carry a worst-case FPR guarantee — the Goswami bound up to a constant or additive term — and differ mainly in cost: Rosetta pays $O(\log_2 \mathcal{L})$ Bloom probes per query, Grafite a single predecessor probe within an additive $2n$ bits, and Memento adds dynamicity while matching Grafite on static workloads. SuRF and SNARF trade that guarantee for shape: SuRF is not parameterized by ε at all and inherits its FPR from the trie skeleton (empirically up to 25% on correlated workloads); SNARF goes below the Goswami bound when its CDF model fits the data and loses the guarantee otherwise. None of the five exploits the cluster-and-gap structure common in real LSM workloads. We keep the worst-case guarantee of the Grafite line and add a distribution-aware mechanism that beats the bound on realistic inputs.

Contributions of this thesis. We start from Goswami's two-layer construction — locality-preserving hash followed by an ERE backend — and extend it in multiple areas, producing a filter that matches the Goswami lower bound on adversarial inputs and beats it on realistic ones.

1. **Single-vector ERE backend (Chapter 2).** We adopt the single-vector D layout (also used by Grafite [?]) in place of Goswami's two metadata vectors D_1, D_2, and back it

with a Poisson bucket-occupancy model that predicts the resulting space saving — 24% relative to the vanilla Goswami backend, confirmed empirically (Section 1.1 sketches the empirical reasoning that justifies this collapse).

2. **Adaptive bucket search (Chapter 2).** We implement Goswami's Weak Prefix Search (WPS) index in full and show empirically that adaptive binary/linear search on packed suffixes outperforms it at the bucket sizes that arise at per-SSTable scale (Section 6.2), removing 59 bits of auxiliary metadata per key and reducing per-query latency by 13–30× relative to WPS.

3. **Truncation hash (Chapter 4).** For sparse inputs we replace Goswami's locality-preserving rotation with the truncation map $h(x) = \lfloor x/2^t \rfloor$, which saves $\log_2 \mathcal{L}$ bits per key.

4. **Distribution-aware segmentation: Seg-ARE (Chapter 5).** A single-pass density-based segmentation partitions the input into dense clusters — handed to exact-mode ERE sub-filters (FPR = 0) — and sparse tails routed to the truncation hash. On SOSD datasets this drives the FPR below 10^{-8} at ~ 11 bits per key on Facebook User IDs and 3.7 bits per key on Wiki timestamps, matching or beating the strongest baselines (Chapter 6); Chapter 7 discusses when Seg-ARE wins, ties, and loses, and the Elias–Fano interpretation behind it.

5. **Improvements to the Grafite reference implementation.** Beyond designing our own filter, we deeply analysed the closest prior art in the Goswami line and patched it with two fixes applied in every measurement of Chapter 6: a balanced-descent replacement for the `select_zero` pathology in the SUX succinct-data-structures library [**?**] on clustered inputs (three-orders-of-magnitude latency regression in the unpatched code, Section 2.1.7); and a lossless Elias–Fano fallback that activates when the upstream constructor would otherwise throw on configurations where lossless storage is cheaper than the requested approximate filter (Section 4.4.2). Grafite is therefore reported in its best configuration throughout this work.

Roadmap.

- **Chapter 2** develops the single-vector ERE and the adaptive bucket-search backend (contributions 1–2).
- **Chapter 3** characterises the key distributions we target, preparing the ground for contributions 3–4.
- **Chapter 4** formalises and compares the SODA and Truncation hash designs (contribution 3).
- **Chapter 5** introduces the Seg-ARE segmentation (contribution 4).
- **Chapter 6** reports the empirical comparison against SuRF, SNARF, Rosetta, and Grafite on real world SOSD and synthetic workloads.
- **Chapter 7** summarises the results of the thesis.

1.4 Notation

General. We identify L-bit keys with the integers $\{0, 1, \ldots, 2^L - 1\}$ via the standard binary representation. Lexicographic comparison of bit strings of the same length coincides with numerical comparison of the corresponding integers, so we use the integer view throughout (this is the natural formulation when the construction performs arithmetic such as rotations or reductions modulo a power of two).

Sets and parameters.

L	Key length in bits.
$U = [0, 2^L)$	Universe of L-bit keys.
n	Number of input keys, $n = \lvert S \rvert$.
$S \subseteq U$	The set of keys in a Range Filter, $\lvert S \rvert = n$.
$Y = U \setminus S$	Non-keys (all points not in S).
$\mathcal{L}$	Maximum query range length.
ε	Target false positive rate.

Hash function and fingerprints.

h	Hash function $h \colon U \to U'$.
K	Fingerprint length in bits, $K = \lceil \log_2(n\mathcal{L}/\varepsilon) \rceil$.
$U' = [0, 2^K)$	Mapped universe.
$r = 2^K$	Mapped universe size.
$S' = h(S)$	Fingerprints of the input keys.
$Y' = h(Y)$	Images of non-keys under h.

Metrics.

FPR	False Positive Rate.
BPK	Bits Per Key: total filter size divided by n.

Structures.

ERE	Exact Range Emptiness — zero-error range filter.
ARE	Approximate Range Emptiness — range filter that may have false positives with probability $\leq \varepsilon$.

1.5 Problem Statement

Let $U = [0, 2^L)$ be the universe of L-bit keys (Section 1.4). Given a set $S \subseteq U$ of n keys, a maximum query range length $\mathcal{L}$, and a target false positive rate ε, the *approximate range emptiness problem* asks to build a data structure that answers "$[a, b] \cap S = \emptyset$?" for intervals of length $\leq \mathcal{L}$, with:

- **No false negatives:** if $[a, b] \cap S \neq \emptyset$, the answer is always "not empty".
- **Bounded false positives:** if $[a, b] \cap S = \emptyset$, the answer is "not empty" with probability at most ε.

Let $Y = U \setminus S$ denote all keys not in the filter.

1.5.1 Information-Theoretic Bounds

? establish the following bounds.

Theorem 1 (Lower bound, **?**, §2)**.** *Any data structure solving the range emptiness problem requires at least $n\big(\log_2(\mathcal{L}/\varepsilon) - c\big)$ bits for an absolute constant $c > 0$.*

Remark 1. *The lower bound is worst-case over the input: it holds for every set $S \subseteq U$ with $|S| = n$, with no assumptions on the structure or distribution of S.*

Remark 2. *In practice, real datasets are often more regular. If the input is known to have additional structure, a data structure may use that structure to reduce the required space below the general bound.*

Theorem 2 (Upper bound, **?**, §3)**.** *The lower bound of Theorem 1 is achieved up to an additive $O(n)$ term via two layers:*

1. *A locality-preserving hash (Section 4.1) $h\colon U \to U'$, where $U' = [0, 2^K)$ and $r = |U'| = 2^K$. The hash partitions U into blocks of size r, applies an independent rotation within each block, and then maps the shifted block onto U' by translation. The hash is drawn from a pairwise-independent family $\mathcal{H}$, so for any $h \in \mathcal{H}$ $\Pr[h(x_1) = h(x_2)$ and $x_1 \neq x_2] \leq 1/2^K$.*
2. *An **Exact Range Emptiness (ERE)** (Section 2.1) structure over $S' = h(S) \subseteq U'$: a succinct structure with zero error and space $n \log_2(r/n) + O(n)$ bits, which answers "$[a', b'] \cap S' = \emptyset$?".*

The resulting space is $n \log_2(\mathcal{L}/\varepsilon) + O(n)$ bits, or equivalently $\log_2(\mathcal{L}/\varepsilon) + O(1)$ bits per key — matching the lower bound.

The fingerprint length $K = \lceil \log_2(n\mathcal{L}/\varepsilon) \rceil$ controls accuracy: because $\Pr[h(x_1) = h(x_2)$ and $x_1 \neq x_2] \leq 1/2^K$ for any pair of keys, the overall error probability of the construction is bounded by $n\mathcal{L}/2^K$, which we constrain to be $\leq \varepsilon$. Thus, increasing K by one bit halves the false positive rate. After ERE compression (which subtracts $\log_2 n$ for block indexing), the effective cost per key is $K - \log_2 n = \log_2(\mathcal{L}/\varepsilon) + O(1)$.

1.5.2 The Role of the Hash Function

The hash h projects U down to U'. Under this projection, the keys S map to fingerprints $S' = h(S)$, and the non-keys $Y = U \setminus S$ map to $Y' = h(Y)$.

A false positive occurs when Y' overlaps with S': a query point $y \in Y$ has $h(y) \in S'$, so the structure cannot distinguish it from a real key. We call the pre-images of a fingerprint $x' \in S'$ its **phantom points** — all non-keys $y \in Y$ such that $h(y) = x'$. The fewer phantoms fall within query ranges, the lower the False Positive Rate (FPR).

The ideal hash would map S' and Y' to disjoint regions of U' — zero overlap, zero false positives. In practice this is impossible within the space budget, but the choice of h determines how well S' and Y' separate for a given data distribution. This motivates two directions explored in this work: alternative hash function designs (Chapter 4) and data-adaptive segmentation strategies that bypass hashing entirely on dense regions (Chapter 5).

Chapter 2

Exact Range Emptiness

2.1 Succinct Representation (SODA 2015, §3.2)

The inner layer of every approximate filter is an *Exact Range Emptiness* (ERE) structure: given a universe U' of K-bit keys and a set $S' \subseteq U'$ of n keys, it answers "$[a, b] \cap S' = \emptyset$?" exactly (no false positives or false negatives).

Remark 3. *Throughout this chapter, S' denotes the input to ERE; in the ARE pipeline (Chapter 4), $S' = h(S)$.*

2.1.1 High-Level Structure

To achieve $n \cdot (K - \log_2(n) + O(1)) = n \log_2(r/n) + O(n)$ bits [**?**], the structure divides the universe U' into $B = 2^{\lfloor \log_2 n \rfloor}$ equal-sized blocks. Conceptually, ERE has two levels: a succinct navigation layer over blocks, and a compact local representation of the keys.

All keys inside one block have a common $\log_2 B = \lfloor \log_2 n \rfloor$ bits prefix. Therefore, key suffixes can be stored in the array with $w = K - \lfloor \log_2 n \rfloor$ bits per key. The succinct navigation layer is used to determine block boundaries.

2.1.2 Original ERE Structure

We now describe the original ERE construction of **?** more precisely. It consists of the following parts:

1. D_1 succinct bit vector: stores block occupancy: $D_1[i] = 1$ iff block i contains at least one key.
2. D_2 succinct bit vector: encodes the sizes of non-empty blocks in unary form. If a non-empty block i contains n_i keys, then it contributes a 1 followed by n_i zeros.
3. A_{ds} the array of structures: each data structure consists of:

   The sorted list of suffixes

   A Weak Prefix Search index over those suffixes (Section 2.1.6).

A query range is split into at most three parts: a sequence of complete blocks and up to two partially covered boundary blocks. The complete-block part is answered by the global succinct layer using D_1. The boundary parts are answered by locating the corresponding local structure in A_{ds} via D_1 and D_2, and then checking emptiness inside the block. The local check uses two weak prefix queries (Section 2.1.6) as an $O(1)$ index into the bucket's sorted suffix list; the stored suffixes themselves witness (non-)emptiness via a final comparison.

2.1.3 Local Search in Buckets

The original paper [?] suggests a Weak Prefix Search (WPS) [?] structure based on a Hollow Z-Fast Trie (ZFT) [?] for $O(1)$ worst-case local queries. We use binary search on bit-packed suffixes, with an adaptive fallback to linear scan for small buckets, instead. At the bucket sizes that occur in practice, binary search — or even linear scan on the smallest buckets — is faster than an $O(1)$ ZFT lookup, whose hidden constant factor is large, and additionally avoids the ZFT's metadata overhead. Even the worst-case bucket of k^* keys takes ∼50–55 bytes at $K = 64$ (Table 2.3), fitting in a single cache line, so a linear scan reduces to one sequential memory access. Details are given in Section 2.1.6.

2.1.4 One-Vector Optimization

In the optimized variant, we simplify the navigation layer. Instead of storing occupancy and non-empty block offsets separately in D_1 and D_2, we replace them with a single succinct bit-vector D. Every block i with n_i keys is encoded as a single 1 followed by n_i zeros, whether the block is empty or not. In particular, an empty block (with $n_i = 0$) is encoded by a single 1. One final sentinel 1 is appended at the end.

Effect on space and navigation. Let M denote the number of non-empty blocks. In the original layout, the metadata size is

$$|D_1| + |D_2| = B + (n + M + 1).$$

In the optimized layout, the metadata becomes

$$|D| = B + n + 1.$$

The exact metadata reduction is M bits. Operationally, this means that the old two-stage lookup through D_1 and D_2 is replaced by direct navigation via SELECT on D.

Metadata footprint on uniform input. Table 2.1 reports the metadata BPK on n uniform 64-bit keys for $n \in \{2^{20},\ 2^{24},\ 2^{28}\}$. The information-theoretic delta is ∼0.63 BPK, matching the analytic prediction $M/n \to 1 - e^{-1} \approx 0.632$ (each block is independently empty with probability e^{-1}, Section 2.1.5). The physical delta is ∼0.80 BPK, amplified by a constant ∼26% overhead from the rank/select index — intrinsic to succinct bit vectors and required for the $O(1)$ navigation guarantees. Both deltas are stable across n.

	Num (logical bits)		**Alloc** (with rank/select index)	
n	two-vector	one-vector	two-vector	one-vector
2^{20}	2.63	2.00	3.33	2.53
2^{24}	2.63	2.00	3.33	2.53
2^{28}	2.63	2.00	3.33	2.53
Δ	-0.63 BPK (-24%)		-0.80 BPK (-24%)	

Table 2.1: Metadata BPK for the two-vector (D_1, D_2) layout (Section 2.1.2) versus the one-vector D layout, on n uniform 64-bit keys. *Num*: logical bitvector length (D_1.NUM()+D_2.NUM() vs. D.NUM()); matches the analytic $|D_1|+|D_2| = B+(n+M+1)$ and $|D| = B+n+1$ formulas above. *Alloc*: physical succinct-bit-vector footprint (bits plus pointer, rank, and select indices). The suffix array A_{ds} is excluded — identical in both layouts.

Against the $\sim$ 20 BPK total ERE footprint — dominated by the suffix array A_{ds}, which is identical between the two layouts — the absolute saving of 0.80 BPK looks small. But it is a $\sim$ 24% reduction in metadata: the only part that can shrink without information loss. Cross-distribution measurements are deferred to Chapter 6 (Table 6.2).

Effect on query operation count. The single-vector layout does not strictly reduce the number of RANK/SELECT calls in every case: the original two-vector layout could short-circuit empty boundary blocks via a D_1.BIT() probe, while the new layout always pays at least two SELECT(D) per boundary block. However, in the two most common ARE query patterns — short ranges that hit a single non-empty block, or that span two adjacent non-empty blocks — the new layout saves *one* and *three* RANK/SELECT calls per query, respectively, with long ranges that scan both boundary buckets saving up to four. The structural advantage is that every RANK/SELECT now hits one contiguous succinct bit vector D instead of alternating between D_1 and D_2, so the auxiliary index pages and cache lines loaded by the first call are likely to serve subsequent calls in the same query. The full per-case breakdown is given in Table 2.2; end-to-end query latency on uniform and clustered data is reported in Chapter 6 (Table 6.4).

Query type	(D_1, D_2) ops	D ops	Δ	Note
Same block, non-empty	$1\,R + 2\,S = 3$	$2\,S = 2$	$\mathbf{-1}$	**Most frequent, hot path**
Same block, empty	0	$2\,S = 2$	$+2$	Cold-path regression
Adjacent, both non-empty	$2\,R + 4\,S = 6$	$3\,S = 3$	$\mathbf{-3}$	**Most frequent, hot path**
Adjacent, both empty	0	$3\,S = 3$	$+3$	Cold-path regression
Long range, both boundaries non-empty	$4\,R + 4\,S = 8$	$4\,S = 4$	-4	Op count halved
Long range, intermediate non-empty	$2\,R = 2$	$4\,S = 4$	$+2$	More ops, single vector
Long range, all empty	$2\,R = 2$	$4\,S = 4$	$+2$	Cold-path regression

Table 2.2: Number of RANK/SELECT operations per query case for the original two-vector (D_1, D_2) layout (Section 2.1.2) versus the one-vector D layout (Section 2.1.4). R denotes RANK on D_1; S denotes SELECT on D_2 or D as appropriate. Bolded rows are the dominant query patterns in typical ARE workloads. BIT probes and bucket-scan operations are not counted (identical between layouts).

Relation to Elias-Fano coding and Grafite. The single bitvector D is equivalent to Elias-Fano coding [**??**]: D is exactly the unary-encoded high-bits bitvector, and the packed suffix array A_{ds} plays the role of the low-bits array. We arrived at this layout by replacing WPS with adaptive search: Section 2.1.6 shows that adaptive linear or binary search over packed suffixes is 13–30× faster (Table 2.5), making the two-vector (D_1, D_2) layout — which exists to support the WPS access pattern — redundant. Grafite [**?**] independently uses the same Elias-Fano structure as their succinct storage layer, motivating the choice by implementation simplicity; our contribution is the empirical quantification of the speed gap (Table 2.5).

2.1.5 Block sizes and Lookup strategy

Block occupancy under uniform distribution. The Exact Range Emptiness structure divides the universe into $B = 2^{\lfloor \log_2 n \rfloor}$ blocks; each non-empty block stores its keys' suffixes in a packed array (a *bucket*). The expected number of keys per block is $\lambda = n/B \in [1, 2)$. When n is a power of two, $B = n$ and $\lambda = 1$ exactly. Each key falls into a uniformly random block, so the occupancy of a fixed block follows $\mathrm{Bin}(n, 1/B)$, which converges to $\mathrm{Poisson}(\lambda)$ as $n \to \infty$ [**?**, Ch. 5].

For $\lambda = 1$, 36.8% of blocks are empty. Among non-empty blocks, the size distribution is sharply concentrated on the smallest values:

- $\Pr[X = 1 \mid X \geq 1] = e^{-1}/(1 - e^{-1}) \approx 58.2\%$ (median of 1 key);
- $\Pr[X \leq 2 \mid X \geq 1] \approx 87.3\%$;
- $\Pr[X \leq 3 \mid X \geq 1] \approx 97.0\%$ ($P_{90} = 3$).

That is, the local lookup is at most a 3-element scan in 97% of non-empty buckets, with 58% degenerating to a single comparison.

The classical balls-into-bins analysis [**?**, Lemma 5.1] bounds the maximum load by $3 \ln n / \ln \ln n$ with probability $\geq 1 - 1/n$. At $n = 2^{30}$ this gives ≈ 21 keys per bucket. Table 2.3 reports the empirical maxima alongside the Poisson reference k^* — the smallest k with $B \cdot \Pr[\mathrm{Poisson}(1) \geq k] < 1$; all measured maxima are well below the Lemma 5.1 bound. Each observed max is the largest bucket over a single realisation of n uniform random 64-bit keys, taken across all B blocks.

n	empty (%)	observed max	k^*	$B \cdot \Pr[X \geq k^*]$	w	k^*-bucket footprint (B)
2^{20}	36.75	9	10	0.12	44	55
2^{24}	36.79	9	11	0.17	40	55
2^{27}	36.79	10	12	0.11	37	56
2^{30}	36.79	13	12	0.89	34	51

Table 2.3: Bucket statistics for n keys in $B = n$ bins ($\lambda = 1$), assuming a full 64-bit universe ($K = 64$). k^*: smallest k with $B \cdot \Pr[\mathrm{Poisson}(1) \geq k] < 1$. $w = K - \lfloor \log_2 n \rfloor$ is the suffix width. The last column gives the bit-packed size of a bucket of k^* keys ($k^* w/8$ bytes, rounded up). In ARE deployments, K and hence w are typically smaller, so the resulting buckets are even more compact.

The Poisson probability mass function $\Pr[X = k] = e^{-\lambda}\lambda^k/k!$ [**?**, eq. (6.1)] makes the tail decay like $1/k!$: even at $B = 2^{30}$, the expected number of blocks with ≥ 20 keys is $\sim 1.7 \times 10^{-10}$,

making buckets larger than 20 keys virtually impossible for uniform data. Such bucket sizes are well suited to linear search.

Non-uniform distributions and worst-case bounds For uniform data the previous paragraph showed that bucket sizes concentrate around $\lambda = 1$: empirical maxima stay below $\sim$13 keys even at $n = 2^{30}$ (Table 2.3), regardless of n.

Real-world data is rarely uniform. The original ARE construction [**?**] uses a *locality-preserving* hash: consecutive input keys map to consecutive hashed values, so dense input regions stay dense in the hashed universe. On heavily clustered benchmarks the heaviest bucket reaches $\sim 6 \times 10^4$ keys. On Wikipedia timestamps at $\mathcal{L} = 16$, median bucket sizes are in the low thousands; near production P90 ($\mathcal{L} = 256$) they grow into the tens of thousands (Table 6.5, Chapter 6). The worst-case footprint stays at $\sim$120 KB — comfortably L2-cache-resident.

Regardless of distribution, the bucket size is hard-bounded above by the block capacity 2^w, where $w = K - \lfloor \log_2 n \rfloor$ is the suffix length: each bucket physically holds at most 2^w distinct w-bit suffixes. The choice of w is constrained by the overall filter size: the total ARE filter occupies approximately $n \cdot (w + 2)$ bits (w bits of suffix data plus $\sim$2 BPK of metadata). Compared to honest storage of 64-bit keys ($64n$ bits), the filter is meaningfully smaller only for w well below 64. In published range filter evaluations the total per-key budget falls in the 10–20 BPK range, with 14–16 BPK as the typical headline configuration (Table 6.1, Chapter 6). After subtracting the $\sim$2 BPK of rank/select metadata, this corresponds to a suffix width $w \in [12, 15]$ bits in our pipeline. A side benefit of $w = 16$ is that 16-bit suffixes align with SIMD lanes for vector-friendly comparisons.

At $w = 15$ the hard upper bound is $2^{15} = 32\,768$ keys per bucket. On a fully packed bucket of 2^w keys binary search terminates in at most $\log_2(2^w) = w$ iterations — and at $w = 15$ this still costs only $\sim$48 ns on Apple M4 Max, comparable to a single Rank/Select on the metadata bit vector. The bound is distribution-independent and depends only on w, not on n. In real workloads observed buckets are typically two orders of magnitude smaller than this hard limit; the worst case is bounded but rare.

Distributions with dense ranges are handled by the hybrid filter of Chapter 5: each cluster is isolated into its own exact sub-filter, turning local density into a compression advantage.

Search strategy. We implement both binary search $O(\log k)$ and linear scan $O(k)$ over the k suffixes of a bucket, and pick at query time based on k. Apple M4 Max benchmarks on uniform random w-bit suffixes: linear scan is fastest for small buckets ($\sim$3 ns at $k = 12$, $\sim$5 ns at $k = 24$, $\sim$10 ns at $k = 64$), while binary search stays cheap even for pathologically large buckets ($\sim$40 ns at $k = 2^{12}$, $\sim$60 ns at $k = 2^{17}$) — logarithmic growth compresses the worst case to tens of nanoseconds.

The default adaptive dispatch (linear for $k \leq 128$, binary above) gives the best of both branches: near-optimal in the common case ($\sim$3 ns at $k \approx 12$) and bounded at tens of nanoseconds even on the largest buckets ($\sim$40 ns at $k = 2^{12}$, $\sim$60 ns at $k = 2^{17}$).

For uniform 64-bit keys with non-empty queries, the total ERE query takes $\sim$33–36 ns on Apple M4 Max, stable across $n \in [2^{20}, 2^{28}]$. Bucket search contributes only $\sim$3 ns at the typical $k \approx 12$; the remainder is Rank/Select navigation on the metadata bit vectors.

2.1.6 Weak Prefix Search

Definition. A *weak prefix query* on a sorted set $S' \subseteq U'$ is specified by a bit string p of length at most $\log_2 |U'|$ and returns the interval of ranks of points in S' whose binary representation has p as a prefix [**?**, §3.2]. If no such points exist, the answer is arbitrary — hence *weak*. The supporting index answers the query in $O(1)$ time in the word-RAM model.

Use in ERE. Given a query range $[a, b]$ inside a single block, the original construction [**?**, §3.2] reduces local emptiness to two weak prefix queries. Let p be the longest common prefix of the binary representations of a and b (an $O(1)$ most-significant-bit operation). Then $S' \cap [a, b] \neq \emptyset$ iff at least one of the following holds:

- the largest point in S' prefixed by $p \cdot 0$ exists and is $\geq a$, or
- the smallest point in S' prefixed by $p \cdot 1$ exists and is $\leq b$.

Each side is one WPS query plus a comparison, giving $O(1)$ local queries. The index is realised as a Z-Fast trie keyed by a Monotone Minimal Perfect Hash Function [**?**].

Why we don't use it. The $O(1)$ guarantee is in the word-RAM model where hash table lookups and trie navigation cost $O(1)$ and space is expressed up to $O(n)$ additive terms. The hidden constants dominate in practice.

We implemented three WPS variants, pairing a Z-Fast trie (standard or succinct) with a Monotone Minimal Perfect Hash Function (MMPHF; fast or compressed):

	Space	Query latency (ns) at $n =$				
Variant	(bits/key)	2^{10}	2^{13}	2^{15}	2^{18}	2^{20}
Baseline (standard trie + fast MMPHF)	179	307	515	557	580	575
Compact (succinct trie + fast MMPHF)	151	490	681	712	751	812
Learned (succinct trie + learned MMPHF)	59	544	683	758	787	889

Table 2.4: Three WPS variants on Apple M4 Max, $K = 64$, 64-bit keys. Query latency generally rises with n because the trie deepens, so each query traverses more levels and pays one hash-probe-and-pointer-chase per level.

Even the most compact variant costs 59 BPK — and this is an auxiliary index *on top of* the packed suffix array, which WPS accelerates but does not replace. In our use case (ERE inside an ARE filter, Section 4.1), the suffix width w is typically 7–13 bits per key depending on the target query range length $\mathcal{L}$, so WPS adds 5–8× more space than the suffix data it indexes.

Query latency is equally problematic. WPS replaces only the local search inside a bucket; Rank/Select navigation on the metadata bit vectors (∼27 ns) is identical for both approaches. Table 2.5 compares the bucket-search step alone across representative bucket sizes.

Bucket size k	Binary search (ns)	WPS Compact (ns)	Speedup
$\sim$12 (typical, uniform)	5–9	490–812	54–160×
2^{12} ($k \approx 4{,}000$)	$\sim$40	490–812	12–20×
2^{17} ($k \approx 131{,}000$)	$\sim$60	490–812	8–14×

Table 2.5: Bucket-search latency on Apple M4 Max, $K = 64$, 64-bit keys. WPS Compact numbers are the full range over $n \in \{2^{10}, \ldots, 2^{20}\}$ from Table 2.4; binary search numbers are from the bucket-scan benchmark of Section 2.1.3. WPS pays a large per-call constant from trie pointer-chasing and hash probing that does not amortise with k, while binary search on packed suffixes scales as $O(\log k)$ with small constants on contiguous memory. For LSM-typical bucket sizes (a few thousand keys) the speedup is 13–30×; the full 8–160× span above covers the range of bucket sizes a generic ARE filter can encounter.

Even on adversarial bucket sizes WPS stays at least 8× slower for the operation it replaces.

The root cause is that the $O(1)$ theoretical guarantee hides large constants: each "constant-time" trie step hashes a variable-length prefix, probes a hash table, and chases a pointer. The probe and the pointer chase are random memory accesses. A WPS query traverses several trie levels and pays this cost on each, whereas a linear scan over a $\leq$13-element bucket ($\leq$55 B at $K = 64$, Table 2.3) touches a single cache line. Binary search on packed suffixes also requires no auxiliary index, whereas the most compact WPS variant adds 59 BPK on top of the suffix data.

2.1.7 Improvements to the Grafite Backend

The same rank/select-and-Elias–Fano machinery that underlies our ERE backend (Section 2.1.4) is also used by Grafite [**?**] as the storage layer of its approximate range filter. While running Grafite on clustered inputs we identified a performance pathology in its predecessor path and contributed a fix that we apply in every measurement of Chapter 6.

The SUX `selectZero` pathology on clustered inputs. Grafite's predecessor query routes through the `select_zero` primitive of the SUX succinct-data-structures library by **?**. SUX answers `select_zero`(i) in two stages: a precomputed *inventory* array stores the position of every fixed-stride zero, giving a coarse jump to the neighbourhood of the answer; from there the original implementation scans the bit vector forward one 64-bit word at a time, subtracting the popcount of each word from the remaining rank until the word containing the i-th zero is found. The forward scan is bounded by the gap between two consecutive inventory entries: $O(1)$ when zeros are spread evenly, but it degrades to a linear walk over many all-ones words when one inventory bucket happens to span a long run of ones — exactly the shape produced by clustered key distributions, where the Elias–Fano upper bit vector alternates between dense regions and large all-ones gaps. On uniform data the primitive costs $\sim$30 ns per query; on SOSD-Wikipedia, where keys form tight near-sequential clusters, the same primitive blows up to $\sim$88 μs per query — a three-orders-of-magnitude regression purely from input geometry.

Our fix adds a second-level *spatial inventory*: an auxiliary array, built at construction time, that stores the cumulative zero count at every 1024-bit boundary of the bit vector. Inside `select_zero`, after the coarse inventory jump, we binary search this array for the 1024-bit block whose cumulative count straddles the target rank, jump directly to that block, and only then run the original word-by-word scan — which is now bounded by a single 1024-bit window regardless

of how the zeros are distributed globally. The query cost becomes $O(\log B + 1024/64)$ with B the number of spatial blocks, replacing the unbounded forward walk. Each spatial-inventory entry is a `uint64` covering 1024 bits of the underlying vector, so the auxiliary structure costs $64/1024 = 1/16$ bit per bit-vector bit; on the Elias–Fano upper vector ($\sim 2n$ bits for n keys) this is $\sim$0.125 BPK of additional space. We patched the SUX sources locally in our CGo wrapper.

2.1.8 Space Analysis

For K-bit keys, ERE stores $w = K - \lfloor \log_2 n \rfloor$ bits of suffix per key plus $\sim$2.53 BPK of metadata on the one-vector layout (single bit vector D with rank/select indices, Section 2.1.4; physical figure from Table 2.1, stable across n). At $K = 64$ and $n = 2^{28}$ ($w = 36$) this gives $\sim$39 BPK total — a $\sim$39% reduction over honestly storing the full 64-bit keys (64 BPK), but still $\sim$2.5$\times$ above the 14–16 BPK budget of industrial range filters (Table 6.1, Chapter 6).

Space grows linearly with K — unavoidable for an exact structure, since it must store the information distinguishing the keys. This linear dependence motivates the move to approximate range emptiness: by storing hashed fingerprints of length $K \approx \log_2(\mathcal{L}/\varepsilon)$ instead of full keys, the space becomes independent of the original key length.

2.1.9 Complexity Summary

Here W denotes the machine word size.

- **Build:** $O(n \cdot K)$ — a single pass over sorted keys.
- **Query:**

  Expected (uniform data): $O(\frac{K}{W})$ — block index lookup dominates; bucket size is $O(1)$ on average (Section 2.1.5).

  Worst case (all keys collide in a single block): $O(\frac{K}{W} \log n)$.

  Inside ARE with K-bit fingerprints fitting in one machine word ($K \leq W$): reduces to $O(1)$ expected.
- **Space:** $n(K - \log_2 n) + O(n)$ bits, matching the information-theoretic lower bound.

Chapter 3

Key Distributions

3.1 Key Distributions in Practice

The theoretical bounds of Section 1.5.1 hold for any key set. In practice, keys in storage systems follow distributions shaped by the application domain.

3.1.1 Common Industrial Distributions

Keys in LSM-tree storage systems typically exhibit one or more of the following patterns:

Sequential / near-sequential. Auto-incrementing primary keys, monotonic timestamps, log sequence numbers. Keys are approximately evenly spaced with small gaps. The key space is dense – the spread is much smaller than the universe. Among the Search on Sorted Data (SOSD) [**?**] datasets, Facebook (user IDs), Books (popularity keys), and Wiki (edit timestamps) all fall into this category.

Clustered. Keys concentrate in a few dense regions separated by large gaps. Event logs keyed by user ID prefix + timestamp cluster around active users; inactive users leave gaps. Geospatial keys such as S2 cell IDs [**?**] cluster around populated areas, with sparse rural regions. OpenStreetMap (OSM) cell IDs are a representative example: they span nearly the full 64-bit space, yet most keys concentrate in a small fraction of it.

Uniform-like. UUIDs, cryptographic hashes, randomly generated tokens. Keys are spread across the full key space with large, roughly equal gaps.

3.1.2 SOSD Benchmark Datasets

For empirical evaluation, we use real-world datasets from the SOSD (Search on Sorted Data) benchmark [**?**]:

Dataset	n (full)	Character
Facebook	200M	User IDs from Facebook's social graph (uint64).
Books	200M	Amazon book popularity keys (uint32).
Books-800M	800M	Amazon book popularity keys (uint64); the same underlying distribution as Books-200M, extended to a 64-bit key space with 4× more keys.
Wiki	200M	Unix timestamps of Wikipedia article edits (uint64).
OSM	800M	Hierarchical cell IDs covering global OpenStreetMap data (uint64).

Table 3.1: SOSD datasets used in our evaluation. Benchmarks in Chapter 6 use subsets of these datasets (e.g. $n = 2^{24}$)

Remark 4. *All datasets are sorted and deduplicated on load. The Wiki dataset contains many repeated timestamps, so its effective n is smaller than the requested raw count.*

Dataset	n	U [bits]	P10	P50	P90	max	mean
Facebook	≈200M	37	2	28	693	5.7×10^5	387
Wiki	≈90M	29	1	1	3	1.7×10^5	3
Books	≈200M	31	1	2	9	2.0×10^4	5
Books-800M	800M	63	7.1×10^7	2.1×10^9	2.3×10^{10}	9.8×10^{13}	1.2×10^{10}
OSM	800M	64	1.6×10^6	4.5×10^7	1.7×10^9	2.3×10^{17}	1.7×10^{10}

Table 3.2: Consecutive-key gap statistics for SOSD datasets after sort and deduplication. n: effective key count. U: key range in bits ($\lfloor \log_2(\max - \min) \rfloor + 1$). Gaps are differences between adjacent sorted keys.

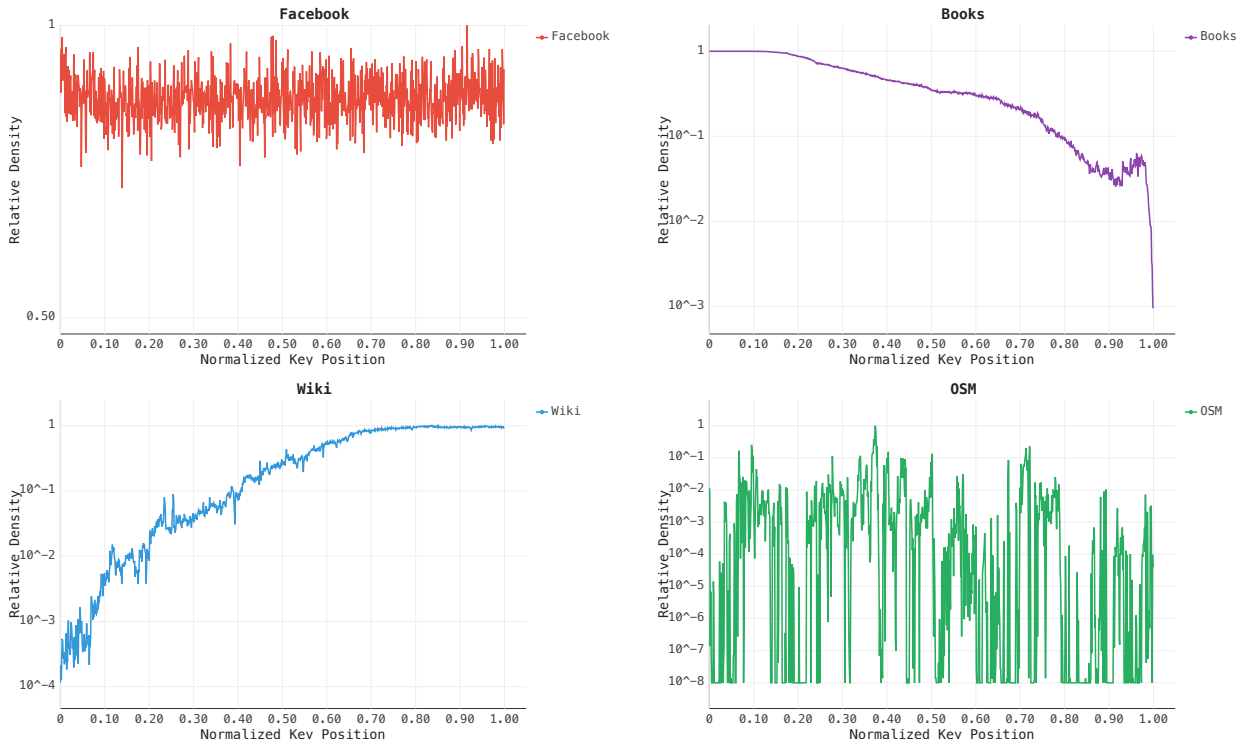

Figure 3.1: Key density histograms for SOSD datasets (1000 bins, log-scale Y-axis). Each bin's value is the fraction of keys falling in it, divided by 1/1000.

3.1.3 Synthetic Distributions

To isolate the effect of specific distribution properties, we also generate synthetic key sets:

- **Uniform:** keys drawn uniformly from $[0, 2^{60})$. Average gap $\approx 2^{60}/n$. No clustering.
- **Clustered:** a mixture of Gaussian clusters with controlled parameters (number of clusters, intra-cluster spread, inter-cluster gap). Benchmarks use 5 to 20 *variable-size* clusters with 15% uniform noise. Cluster centers are drawn uniformly at random from $[0, 2^{64})$, so the expected gap between adjacent centers is $\approx 2^{64}/\text{numClusters}$. Each cluster's standard deviation σ is drawn uniformly from $[2^{20}, 2^{30})$ and floored at the cluster size.

Together, the SOSD and synthetic datasets span the spectrum from near-sequential (Facebook, Books, Wiki) through clustered (OSM) to structureless (uniform random) — allowing us to evaluate each filter design across the range of distributions it may encounter in practice.

3.1.4 An assumption on queries

We assume that queries target regions of the key space where stored keys are dense, rather than arbitrary empty regions.

Assumption 1. *Queries reflect the application state and therefore concentrate on regions that contain data.*

Consequently, FPR in sparse regions of the key space matters less — dense regions dominate the effective FPR experienced by the application.

The SODA hash (Section 4.1) ignores this assumption entirely — it provides worst-case guarantees for any distribution. The truncation hash (Section 4.2) performs well only under uniformity. The hybrid approaches (Chapter 5) explicitly exploit it by isolating dense regions.

Section 3.2 operationalizes this assumption in the query workload.

3.2 Query generation

Queries are drawn from a three-bucket mixture that reflects the assumption of Section 3.1.4:

- **50% near-key.** A random stored key k is chosen; the query start is drawn uniformly from $[k - 5L,\ k + 5L)$. Models application lookups that miss by a small margin.
- **30% in-gap.** A random gap of size ≥ 2 is selected uniformly; the query is placed inside it and truncated to the gap boundary if it overflows. Empty by construction on every dataset. Models misses in locally dense regions.
- **20% uniform.** A random start is drawn uniformly from $[\min k, \max k]$. Covers sparse regions to verify FPR stays reasonable there too.

The near-key and in-gap buckets reflect the dominant workload; the uniform bucket acts as a safety check on sparse regions. The headline *gap-heavy* workload of Section 6.5 reweighs the mixture to 0 / 70 / 30 (no near-key, 70% in-gap, 30% uniform) so that no candidate query is guaranteed empty just by its construction.

The two metrics this thesis cares about — FPR and query latency — want different things from the same query stream, so we draw two query sets from the mixture above and let truncation decide which.

3.2.1 Empty queries for False Positive Rate Measurement

For FPR estimation every measured query must be empty: a positive answer on a non-empty query is a true positive, not a false one, and would deflate the FPR estimate. We enforce emptiness by *truncation*: when a candidate query $[a, a + L)$ would contain a stored key k^*, it is shortened to $[a, k^* - 1]$ rather than discarded and resampled. Truncation preserves queries that start inside a cluster and end between two closely spaced keys, which would otherwise be rejected every time — biasing the workload towards cluster boundaries and missing the harder case of a query landing deep inside a dense region. The effective range length may therefore be smaller than L.

3.2.2 Combined queries (for latency)

In a deployed LSM-tree, the primary performance contribution of a filter lies in its handling of empty queries. When a filter returns a "not empty" result, the storage engine must proceed with a block read, an I/O operation whose cost typically dominates the filter's execution time. Thus, the filter's latency is most critical when it successfully prevents a disk fetch. For this reason, our primary latency evaluations are conducted on the same empty-only query stream used for FPR measurement.

Nevertheless, it is still necessary to ensure that filter performance does not degrade severely on non-empty queries. If a filter's "not empty" evaluation path were significantly slower than its "empty" path, it could introduce noticeable CPU overhead before the I/O operation even begins. To evaluate this, we generate a second query stream using the same mixture, but omit the forced truncation step (while retaining in-gap truncation). In this stream, a query that overlaps with a stored key is evaluated at its full length L rather than being artificially shortened to $[a, k^* - 1]$.

The non-empty fraction depends on the local key density and on L. Under the smart-mix weights (50 / 30 / 20) the 30% in-gap queries remain empty by construction; the other two buckets contribute non-empty queries as follows. The 50% near-key bucket draws an offset uniformly from $[-5L, 5L)$, so even on a sparse universe roughly $\frac{1}{10}$ of these queries land on the reference key — a $\sim$5% non-empty floor that is independent of data density. On distributions where the average gap between adjacent keys is smaller than L, near-key queries also catch one or more neighbours and the bucket saturates near its 50% limit. The 20% uniform bucket adds about $L \cdot n/U$ on top, which matters on small-universe inputs (Facebook, Wiki) and is negligible on 64-bit universes (OSM, Books-800M, Uniform). The ceiling is therefore 70% when both buckets always hit. Table 3.3 reports the measured fractions at $n = 2^{28}$ across the datasets used in Section 6.5.

Dataset	$L = 1$	$L = 16$	$L = 1024$
Facebook	9.0%	20.7%	50.6%
Wiki	46.3%	63.2%	68.4%
Books-800M	5.0%	5.0%	4.9%
OSM	5.0%	5.0%	4.9%
Uniform	5.0%	5.0%	4.9%
Clustered	4.2%	6.8%	9.7%

Table 3.3: Non-empty fraction of the combined-queries workload under the smart-mix weights ($n = 2^{28}$). Wiki has average inter-key gap close to 1, so its near-key bucket is essentially fully non-empty already at $L = 1$. The other dense distribution, Facebook, picks up the uniform-bucket contribution as L grows. The large-universe datasets (OSM, Books-800M, Uniform) sit at the 5% near-key floor; the synthetic Clustered set is only mildly denser.

Chapter 4

Hash Function Approaches

4.1 Pairwise-Independent Hash (SODA Baseline)

4.1.1 The Hash Function

The paper's original locality-preserving hash [**?**, §3.1]:

$$h(x) = \big(u(\lfloor x/2^K \rfloor) + (x \bmod 2^K)\big) \bmod 2^K \tag{4.1}$$

That is: compute the block index $\lfloor x/2^K \rfloor$, hash it with u to get a per-block rotation, then add the key's within-block offset $x \bmod 2^K$. The result is in $[0, 2^K)$.

$u : [0, U/2^K) \to [0, 2^K)$ is drawn from the multiply-add-shift family of **?** (a power-of-two-modulus variant of the pairwise-independent construction of **?**), stored as two 64-bit coefficients (a, b):

$$u(x) = \left\lfloor \frac{a \cdot x + b \bmod 2^{64}}{2^{64-K}} \right\rfloor . \tag{4.2}$$

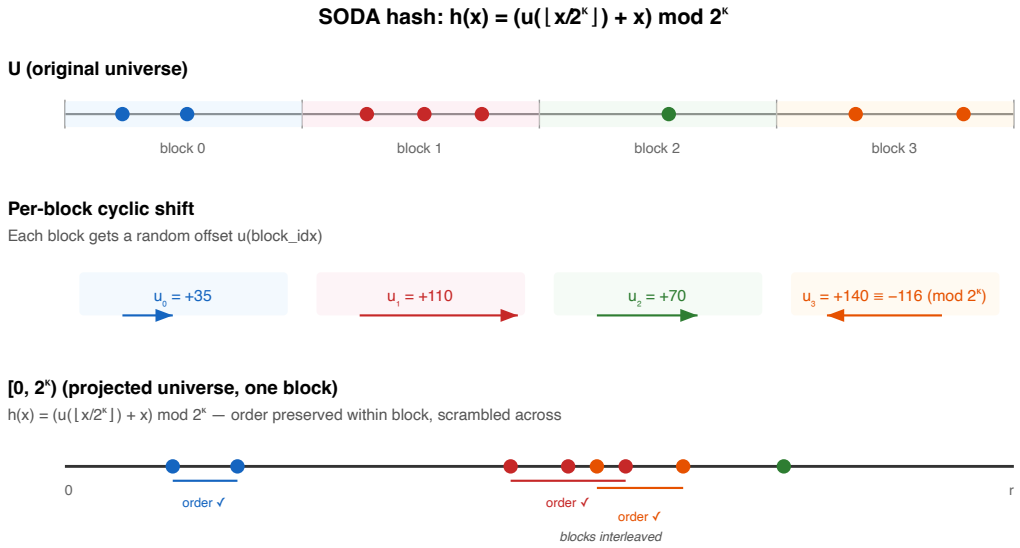

Figure 4.1: SODA hash mechanism: per-block rotation maps keys to $[0, 2^K)$. A query range spanning two blocks produces at most two intervals in the mapped space.

Properties of the hash:

- **Pairwise-independent:** for any $h \in \mathcal{H}$ from the multiply-add-shift family (Equation (4.2)), $\Pr[h(x_1) = h(x_2) \text{ and } x_1 \neq x_2] \leq 2^{-K}$. This holds for sequential, clustered, or adversarial data.
- **Locality-preserving:** the hash applies a per-block rotation, so consecutive keys within the same block map to consecutive positions in $[0, 2^K)$. When $\mathcal{L} \leq 2^K$, a query range $[a, b]$ spans at most 2 blocks, producing at most 2 intervals in $[0, 2^K)$, each checked with one IsEmpty call to ERE. (The choice $K = \lceil \log_2(n\mathcal{L}/\varepsilon) \rceil \geq \log_2 \mathcal{L}$ guarantees this.)

Comparison with Grafite. Grafite [**?**] uses a hash of the same locality-preserving form but draws its per-block shift from a different pairwise-independent family — multiply-add-modulo over a Mersenne prime, requiring true modulo operations, whereas the multiply-add-shift family of **?** used here only needs multiplications and shifts. The theoretical guarantees are identical; Section 4.4 dissects Grafite's full construction and our implementation-level contributions to it.

4.1.2 ARE Construction With SODA Hash

This is a direct implementation of the construction from **?**, §3. The hash design, its proofs of locality-preservation and pairwise independence, and the resulting space/FPR bounds are all from the original paper — our contribution is the practical implementation.

Combining this hash with ERE (Chapter 2) yields the baseline ARE filter:

1. Divide U into blocks of size 2^K. Let $\text{blockIdx}(x) = \lfloor x/2^K \rfloor$ be the block index of key x.
2. For each block, compute pairwise-independent shift: top K bits of $(a \cdot \text{blockIdx}(x) + b)$.
3. Hash each key via Equation (4.1).
4. Sort, deduplicate, build ERE over $[0, 2^K)$.
5. Query: hash both endpoints; rotation may split the range into two intervals — check both in ERE.

Filter properties.

- **FPR $\leq \varepsilon$ for any data distribution** — follows from pairwise independence of the hash, with $K = \lceil \log_2(n\mathcal{L}/\varepsilon) \rceil$ giving $\text{BPK} = \log_2(\mathcal{L}/\varepsilon) + O(1)$, matching the information-theoretic lower bound. Including ERE metadata, the total cost is $\log_2(\mathcal{L}/\varepsilon) + \sim 2$ bits per key.
- **No false negatives** — if key $x \in [a, b]$, then $h(x)$ lies in the mapped range $h([a, b])$, which is fully checked in ERE.

4.1.3 Empirical Validation

On uniform data ($n = 2^{20}$, $\mathcal{L} = 128$), the measured FPR-vs-BPK curve tracks the theoretical bound with a gap of $\sim$2 BPK (ERE overhead, Figure 4.2).

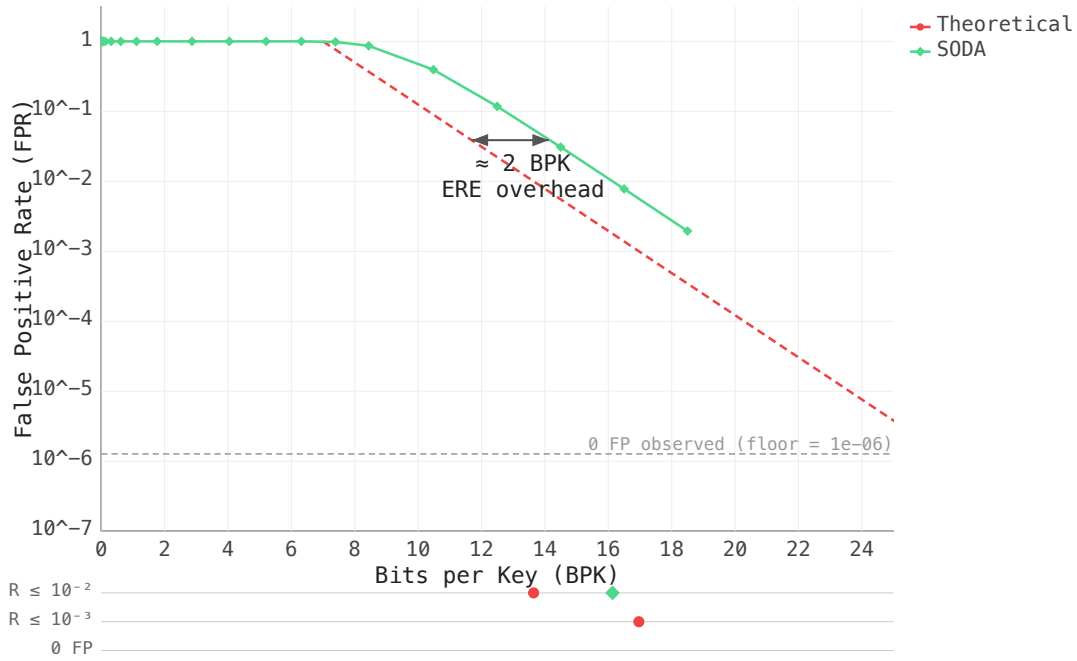

Figure 4.2: FPR vs BPK on uniform keys ($n = 2^{20}$, $\mathcal{L} = 128$). SODA tracks the theoretical bound with ∼2 BPK overhead from ERE metadata.

4.2 Prefix Truncation

4.2.1 The Hash Function

ERE size grows with the bit-length of keys. We need to compress keys into a smaller range $[0, 2^K)$ while preserving locality – so that a query interval maps to a bounded number of intervals in the compressed space. This need not be a hash in the cryptographic sense; the simplest locality-preserving compression is plain bit-truncation:

$$h(x) = \lfloor x/2^t \rfloor,$$

keeping the top $K = L - t$ bits of each key (where L is the key width in bits).

Properties of the hash:

- **Locality-preserving:** truncation preserves order — if $a < b$, then $h(a) \leq h(b)$. Intervals map to intervals.
- **Contiguous phantoms:** recall that the phantom points of a key x are all non-keys y with $h(y) = h(x)$ — the pre-images that cause false positives. Under truncation they all fall within an interval of length 2^t in the original universe: exactly the non-keys sharing the same K-bit prefix as x. This contrasts with the SODA hash, where phantoms are scattered uniformly.

Remark 5. *Throughout this section, $2^t = 2^{L-K}$ is the universe compression factor; the number of original-universe points that hash to the same value in $[0, 2^K)$. For truncation this matches the definition: t bits are dropped from the back of each key.*

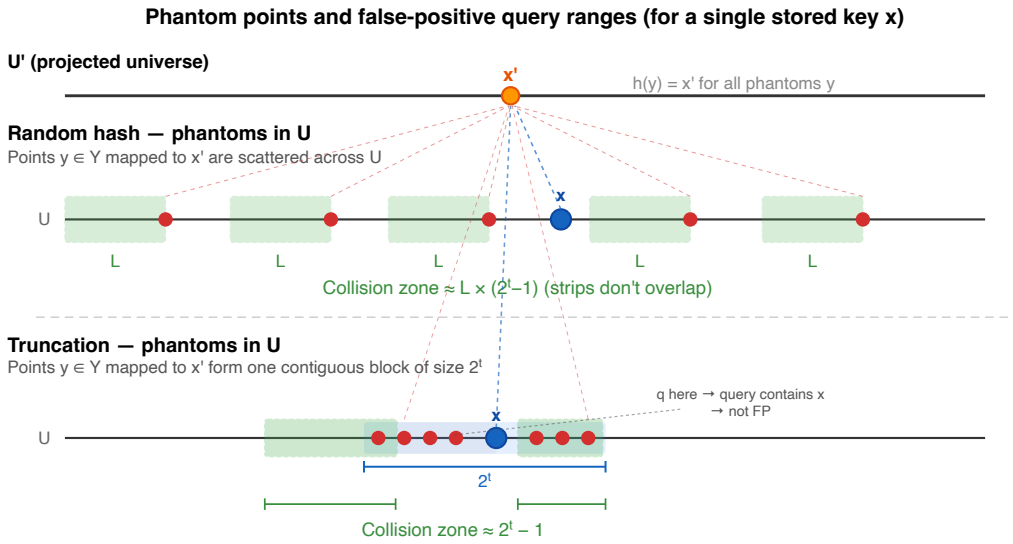

Figure 4.3: Green strips show the **collision zone** of stored key x: starting positions q of queries $[q, q+\mathcal{L}-1]$ that are false positives due to x (i.e. $x \notin [q, q+\mathcal{L}-1]$ yet $h(x) \in h([q, q+\mathcal{L}-1])$). Each scattered phantom of the random hash contributes $\mathcal{L}$ such positions (phantom at the strip's right edge); truncation's contiguous block limits the total to $2^t - 1$, independent of $\mathcal{L}$.

4.2.2 ARE Construction With Truncation

Combining truncation with ERE yields a filter where the contiguous phantom structure (Figure 4.3) changes the collision analysis. We define the **collision zone** $\mathrm{CZ}(x)$ of stored key x as the set of query starting positions q for which $[q, q+\mathcal{L}-1]$ is a false positive due to x:

$$\mathrm{CZ}(x) \;=\; \big\{\, q \;:\; x \notin [q, q+\mathcal{L}-1] \text{ yet } h(x) \in h([q, q+\mathcal{L}-1]) \,\big\}.$$

Its cardinality $|\mathrm{CZ}(x)|$ counts the false-positive starts contributed by x.

- **SODA hash:** each of the 2^t-1 phantom positions is far from x, so all $\mathcal{L}$ starting positions that cover it also exclude x; $|\mathrm{CZ}(x)| = \mathcal{L}(2^t - 1)$ (Section 4.1).
- **Truncation:** all $2^t - 1$ phantoms share one contiguous block with x. A query misses x only when it starts inside $(x, \text{block\_right}]$ or ends inside $[\text{block\_left}, x)$; these two intervals have widths $(2^t - 1 - r)$ and r respectively, so $|\mathrm{CZ}(x)| = 2^t - 1$ regardless of $\mathcal{L}$ (Eq. (4.3)).

The additive (rather than multiplicative) interaction eliminates the $\mathcal{L}$ factor from the filter's space formula:

Hash	$\lvert\mathrm{CZ}(x)\rvert$	K for FPR $\le \varepsilon$	Filter BPK
Random (SODA)	$\mathcal{L} \times (2^t - 1)$	$\log_2(n\mathcal{L}/\varepsilon)$	$\log_2(\mathcal{L}/\varepsilon) + O(1)$
Truncation	$2^t - 1$	$\log_2(n/\varepsilon)$	$\log_2(1/\varepsilon) + O(1)$

Table 4.1: Choosing truncation over the SODA hash reduces the resulting filter's BPK by $\log_2(\mathcal{L})$. At $\mathcal{L} = 128$, this is a saving of 7 bits per key.

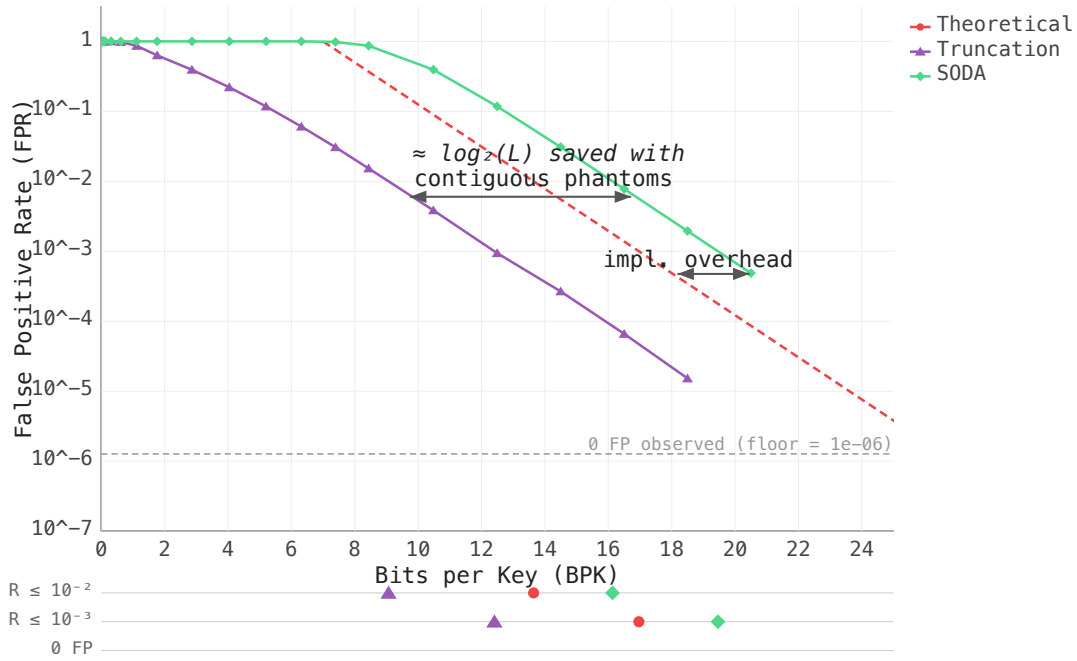

Figure 4.4: FPR vs BPK for truncation vs SODA on uniform keys ($n = 2^{20}$, $\mathcal{L} = 128$). The SODA curve is shifted right by $\approx \log_2(128) = 7$ BPK — matching the theoretical prediction.

4.2.3 The Price of Contiguity

The same contiguity that saves space may also be a weakness in other scenarios.

FPR for Uniform Key Spacing

Let R denote the uniform gap between consecutive keys. Assume keys x_0, $x_0{+}R$, $x_0{+}2R, \ldots$ A query $[q, q{+}\mathcal{L}{-}1]$ is truly empty iff it fits inside a gap: $q = x_i + s$, $s \in [1, R{-}\mathcal{L}]$.

A false positive requires some key's hash value to fall inside $h([q, q+\mathcal{L}-1])$. Since the query is empty, every block *fully covered* by the query is key-free: any key inside such a block would lie in the query and contradict emptiness. Only the two *edge* blocks — the one straddled by q on the left and the one straddled by $q + \mathcal{L} - 1$ on the right — can hold a key whose hash collides with $h([q, q + \mathcal{L} - 1])$, and it suffices to consider the immediate neighbours x_i and x_{i+1}: order-preservation of h gives $h(x_{i+2}) \geq h(x_{i+1})$, so if x_{i+2} collides with the query then x_{i+1} does too — further keys cannot trigger *new* false-positive events (symmetrically on the left). The argument holds regardless of how many blocks the query spans.

Let $\alpha_i = x_i \bmod 2^t$ and $\alpha_{i+1} = x_{i+1} \bmod 2^t$ be the offsets of the neighbours within their respective blocks. A key's **phantom block** is the interval of points sharing the same K-bit prefix as the key — i.e., the 2^t universe positions that truncation maps to the same fingerprint.

- **Left overlap:** the query overlaps x_i's phantom block iff $q \leq x_i + 2^t - 1 - \alpha_i$, i.e., $s \leq 2^t - 1 - \alpha_i$ — contributing $2^t - 1 - \alpha_i$ values of s.
- **Right overlap:** the query overlaps x_{i+1}'s phantom block iff $q + \mathcal{L} - 1 \geq x_{i+1} - \alpha_{i+1}$, i.e., $s \geq R - \mathcal{L} + 1 - \alpha_{i+1}$ — contributing α_{i+1} values of s.

When $R - \mathcal{L} \geq 2^{t+1}$, the two ranges of s are disjoint, so the false-positive count per gap

is $(2^t - 1 - \alpha_i) + \alpha_{i+1}$. Treating α_i, α_{i+1} as uniform on $\{0, \ldots, 2^t - 1\}$ (mean $(2^t - 1)/2$) and dividing by the $R - \mathcal{L}$ empty-query positions:

$$\text{FPR} \;=\; \frac{2^t - 1}{R - \mathcal{L}} \;\approx\; \frac{2^t}{R - \mathcal{L}}. \tag{4.3}$$

Capping at 1 and handling the $R \leq \mathcal{L}$ case:

$$\text{FPR} = \begin{cases} 0 & R \leq \mathcal{L} \text{ (no empty queries)}, \\ \min\bigl(1,\ 2^t/(R - \mathcal{L})\bigr) & R > \mathcal{L}. \end{cases}$$

Dense or sequential keys ($R \leq \mathcal{L}$) have FPR = 0 for free: every query of length $\mathcal{L}$ hits a key, so the filter is exact. When $R > \mathcal{L}$, Eq. (4.3) reads FPR $\approx 2^t/(R - \mathcal{L})$, so the gap excess $R - \mathcal{L}$ has to be a large multiple of the phantom-block width 2^t to keep FPR low. Concretely: at $R - \mathcal{L} = 10 \cdot 2^t$ the FPR is $\approx$0.1; at $R - \mathcal{L} = 2 \cdot 2^t$ it is already $\approx$0.5; at the floor $R - \mathcal{L} \leq 2^t$ the phantom block covers (almost) all of the gap and the formula saturates at FPR = 1.

Breakdown thresholds for SOSD datasets

From Eq. (4.3), high FPR occurs when $2^t \gtrsim R - \mathcal{L}$, so the critical truncation depth is $t^* = \lfloor \log_2(R - \mathcal{L}) \rfloor$. Table 4.2 shows t^* for typical SOSD gaps and several query lengths.

Dataset	Typical R	$\mathcal{L} = 4$	$\mathcal{L} = 16$	$\mathcal{L} = 128$	$\mathcal{L} = 1024$
Wiki / Books	1–5	0*	0*	0*	0*
Facebook P50	28	4	3	0*	0*
Facebook mean	387	8	8	8	0*
OSM P50	4.5×10^7	25	25	25	25

* $R \leq \mathcal{L}$: every query contains a key.

Table 4.2: Critical truncation depth t^* at which FPR $\to 1$ for SOSD gap statistics (Table 3.2) and varying query length $\mathcal{L}$.

Wiki and Books have gaps well below any practical $\mathcal{L}$, so every query hits a key. Facebook's tail (gaps up to $\sim$700) breaks at $t \approx 8$. OSM has $t^* \approx 25$ for all query lengths. In a 64-bit universe this means keeping $K = 64 - 25 = 39$ bits just to avoid complete failure; ERE compresses these to $K - \log n \approx 19$ BPK at $n = 2^{20}$. Achieving FPR $\leq 10^{-3}$ requires $t \leq 15$ ($K \geq 49$, ≈ 29 BPK) under uniform queries. Thus, truncation is impractical for clustered data.

Remark 6. *The derivation assumes* ***queries are placed uniformly at random in the universe****, so α_i, α_{i+1} are uniform on $\{0, \ldots, 2^t - 1\}$. Benchmark queries (Section 3.2) are drawn from the gaps between keys, so they land near phantom block more often than a uniform draw would.*

The SODA hash avoids this entirely: FPR $\leq \varepsilon$ for any key distribution, at the cost of $\log_2(\mathcal{L})$ extra bits per key.

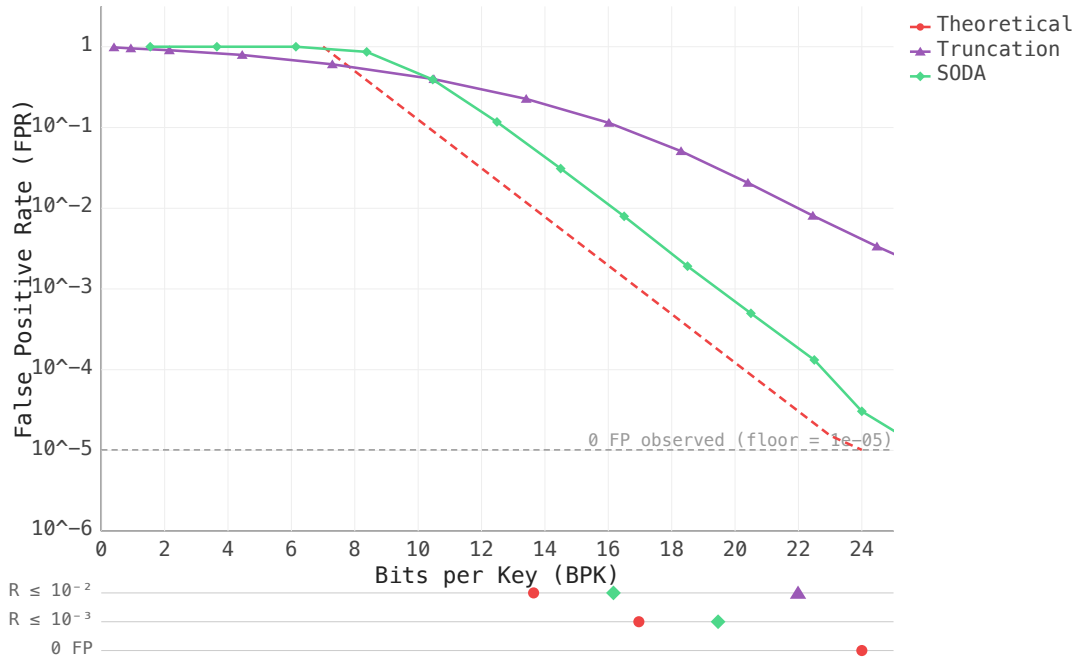

Figure 4.5: FPR vs BPK on OSM ($n = 2^{20}$, $\mathcal{L} = 128$, in-gap queries, Section 3.2). Truncation is bounded below by a structural FPR floor set by the gap-to-2^t ratio (Eq. 4.3): at 24 BPK it remains at FPR $\approx 10^{-2}$ while SODA reaches $\sim 10^{-5}$.

Remark 7. *Grafite [?] independently proposes an equivalent heuristic named* Bucketing *(their Section 4): keys are mapped via $x \mapsto \lfloor x/s \rfloor$ and the occupied bucket indices are encoded with Elias-Fano, with s as the space-accuracy trade-off parameter — the same idea as prefix truncation with $s = 2^t$. Our contribution relative to Grafite is the precise FPR formula (Equation 4.3) and the per-dataset breakdown analysis (Table 4.2), which Grafite does not provide.*

4.3 Adaptive Filter

The SODA hash compresses keys into a K-bit universe of size $2^K = n\mathcal{L}/\varepsilon$. If the original key spread $\max(S) - \min(S)$ is already smaller than 2^K, no hash is needed at all — we build ERE directly on the (normalized) keys and get FPR $= 0$ for free. The adaptive filter checks this condition at build time.

4.3.1 Exact Mode

1. **Normalize:** subtract $\min(S)$ from all keys, shifting the dataset to start at 0.
2. **Measure spread:** $M = \lceil \log_2(\max(S) - \min(S)) \rceil$.
3. **Decide:**
   - $M \leq K$: the entire dataset fits in K bits $\Rightarrow$ **exact mode**. Build ERE directly over normalized keys. No hash, FPR $= 0$.
   - $M > K$: data too spread $\Rightarrow$ **SODA mode**. Apply a hash drawn from a pairwise-independent family, same filter guarantees as Section 4.1.

4.3.2 When Does Exact Mode Trigger?

Rewriting the condition

$$\max(S) - \min(S) < 2^K = \frac{n\mathcal{L}}{\varepsilon}$$

in terms of the average gap:

$$\bar{g} = \frac{\max(S) - \min(S)}{n-1}$$

$$\bar{g} < \frac{\mathcal{L}}{\varepsilon}. \tag{4.4}$$

At $\mathcal{L} = 128$, $\varepsilon = 10^{-3}$ the threshold is 128,000 — well above the Facebook median gap of 28 or the Books mean of 5, so exact mode triggers on these datasets for free.

As we will see in Chapter 5, this becomes especially powerful in combination with Seg-ARE: each dense cluster is isolated into its own adaptive filter, where the local spread is small enough for exact mode, giving FPR = 0 on the dense parts of the data.

4.4 Grafite: A Closer Look

Section 1.2 introduces the Grafite construction: locality-preserving hash, Elias–Fano storage of hash codes, the Theorem 3.4 space bound. We focus here on two angles relevant to this thesis: parameter choices (degradation on long queries and an implicit universe definition that diverges from our Adaptive Filter, Section 4.3), and a lossless fallback path our CGo (C/Go FFI bridge) wrapper installs.

4.4.1 Parameter Choices and Their Consequences

Let ℓ denote the actual query length (which may exceed the configured $\mathcal{L}$) and let B denote Grafite's per-key budget in bits, so that the Elias–Fano fingerprint storage occupies 2^{B-2} slots per key (the constant 2 accounts for the Elias–Fano lower-/upper-bits overhead, see [**?**]). Queries longer than the configured $\mathcal{L}$ degrade as $\min\{1, \ell/2^{B-2}\}$; queries shorter get the design FPR for free. On top of the budget identity, Theorem 3.4 of [**?**] carries the restriction $\mathcal{L} \leq u\varepsilon/n$ — equivalently $r = n\mathcal{L}/\varepsilon \leq u$. Above this point the requested hash range exceeds the universe, and the upstream constructor throws. The authors use this throw deliberately, to direct the user to a lossless encoding (Elias–Fano over the raw keys, or any other exact representation): past this point an approximate fingerprint costs more bits than storing keys exactly, so building the hash is wasteful.

Universe normalization. Grafite measures the universe as $\max(S)$ rather than $\max(S) - \min(S)$: keys are passed unshifted into both the hash and the Elias–Fano builder, and the budget check fires on the absolute maximum, not on the spread. Our Adaptive Filter (Section 4.3) subtracts $\min(S)$ first. When $\min(S)$ is far from zero, the normalized universe is strictly smaller — the Adaptive filter then enters exact mode on a wider parameter range, and the resulting Elias–Fano encoding is smaller once both are in exact mode.

4.4.2 Lossless Elias–Fano Fallback

We make the mode-selection automatic: our CGo wrapper catches the throw and substitutes a direct `grafite::ef_sux_vector` over the deduplicated input keys, giving deterministic FPR = 0 at $n \cdot (2 + \log_2(\max(S)/n)) +$ select-index overhead bits. Grafite then remains comparable across the full budget axis instead of failing on its tightest-ε end. The next chapter generalises this single-segment exact-mode trigger into a per-cluster mechanism via segmentation.

Chapter 5

Segmented ARE Filters

Chapter 3 identified three distribution types. Sequential and near-sequential keys have small local spread, so exact mode (Section 4.3) applies directly — FPR = 0 with no hash needed. When keys are uniform over a large span, truncation (Section 4.2) is effective: large inter-key gaps mean few phantom collisions and low FPR.

Clustered keys (OSM, geospatial IDs) fit neither case alone: dense regions inside each cluster behave like near-sequential data, while the sparse gaps between clusters have large inter-key distances where truncation works well.

The hybrid idea is to split along this structure — isolate each dense region into its own exact-mode sub-filter (FPR = 0), and route the sparse keys between clusters to a fallback filter.

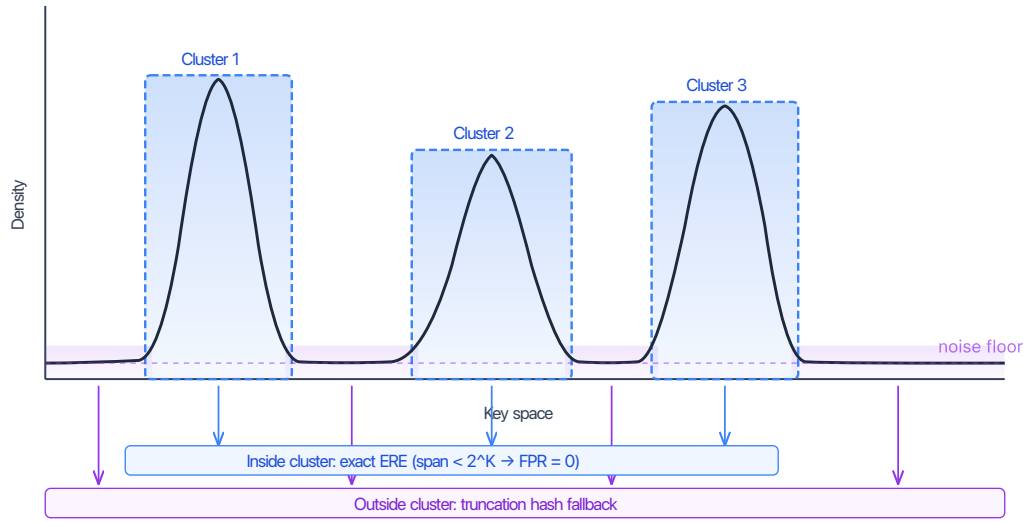

Figure 5.1: Segmented ARE filter: dense clusters go into exact-mode sub-filters (FPR = 0); sparse keys between clusters go to a fallback.

This chapter describes Seg-ARE, our segmentation algorithm based on 1D DBSCAN.

5.1 1D DBSCAN in $O(n)$

DBSCAN [?] is a density-based clustering algorithm. It takes two parameters: a neighbourhood width δ and a density threshold `minPts`. A point is a *core point* if it has at least `minPts` neighbours within a window of width δ; a *border point* if it is within δ of a core point but not itself dense; and *noise* otherwise. Clusters grow from core points by absorbing all density-reachable neighbours.

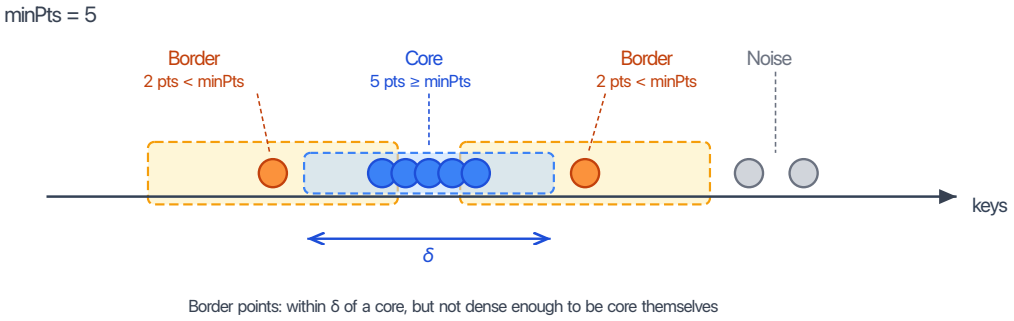

Figure 5.2: 1D DBSCAN on a key sequence (minPts = 5). Blue = core points (window contains $\geq$ minPts neighbours). Orange = border (within δ of a core, but $<$ minPts in own window). Gray = noise.

On sorted one-dimensional data, density queries reduce to a sliding window, giving an $O(n)$ implementation without any auxiliary index.

5.2 Seg-ARE: Design and Algorithm

Border expansion increases bits per key. Standard DBSCAN absorbs border points — points within δ of a core but not themselves dense — into the nearest cluster. This widens the cluster's span without adding proportional density. The exact-mode ERE sub-filter encodes keys relative to that universe, so a wider span means more bits per key. Seg-ARE drops border expansion: the dense core alone minimises span and keeps the sub-filter cost low.

Expressing δ through `minPts`. Standard DBSCAN treats δ and `minPts` as independent. We choose δ small enough that every range produced by the forward sweep fits the exact-mode budget of the underlying ERE sub-filter (see Section 5.2.4 for the derivation):

$$\delta = \texttt{minPts} \cdot \frac{\mathcal{L}}{2\varepsilon} = \frac{256\,\mathcal{L}}{\varepsilon}. \tag{5.1}$$

This ties δ to `minPts`, leaving a single free parameter.

Choosing `minPts` = 512. The choice balances three concerns:

- **Overhead per key.** Each segment costs roughly 700 bits of fixed metadata (`minKey`, `maxKey`, pointer, sub-filter header). At 512 keys this adds $\sim$ 1.4 BPK; for smaller clusters the overhead quickly dominates the budget.

- **Query cost.** The segment count is at most $\lfloor n/\texttt{minPts} \rfloor$, so the binary search over segment boundaries (Section 5.2.3) stays fast regardless of dataset size.
- **Detection power.** If `minPts` is set too high, genuinely dense sub-ranges with fewer keys go entirely to the fallback, negating the benefit of segmentation.

512 balances these concerns: small enough to catch real clusters, large enough to keep per-segment overhead acceptable.

To sanity-check the scale: for $\mathcal{L} = 128$, $\varepsilon = 10^{-3}$ we get $\delta \approx 3.3 \times 10^7$. Comparing this against the SOSD gap statistics (Table 3.2), Facebook/Wiki/Books have median gaps well below δ so most keys fall into segments, while OSM's median gap of 4.5×10^7 sits right at the threshold — segmentation captures only the denser sub-clusters. A uniform 60-bit distribution has average gap $\sim 2^{60}/n \gg \delta$ for every $n \leq 2^{28}$, so every key goes to the fallback and Seg-ARE degrades to plain truncation.

5.2.1 Algorithm

Seg-ARE runs in three steps:

1. **Forward sweep:** slide a two-pointer window. Whenever the window holds $\geq$ `minPts` keys, record the index range [*left*, *right*].
2. **Conditional merge:** adjacent ranges with key-space gap $\leq \delta$ are joined only if the combined span is below $m'\mathcal{L}/\varepsilon$, where m' is the total key count of the merged segment. Otherwise they remain separate.
3. **Fallback:** all keys outside merged ranges go to the fallback filter.

5.2.2 Fallback

Keys not assigned to any segment go to a single fallback ARE filter. Depending on the density of the fallback keys, this may be a truncation filter or a hash-based one.

5.2.3 Query

Segments are sorted by $[\texttt{minKey}, \texttt{maxKey}]$. For a query $[a, b]$:

1. Binary search over segments intersecting $[a, b]$ — $O(\log C)$, C = segment count.
2. Check the first intersecting segment only. Suppose the query $[a, b]$ intersects two segments $S_1 = [\texttt{minKey}_1, \texttt{maxKey}_1]$ and $S_2 = [\texttt{minKey}_2, \texttt{maxKey}_2]$ with $\texttt{maxKey}_1 < \texttt{minKey}_2$. Then $a \leq \texttt{maxKey}_1$ and $b \geq \texttt{minKey}_2$, so the query covers $\texttt{maxKey}_1$ — a real key. The exact-mode sub-filter of S_1 reports non-empty, and no further segments need to be checked.
3. Check the fallback filter.
4. Return "empty" only if all agree.

5.2.4 FPR Guarantee

Every segment sub-filter operates in exact mode (FPR = 0).

Individual range. Consider a range of m sorted keys $k_0 < k_1 < \cdots < k_{m-1}$ from the forward sweep. The window constraint says any `minPts` consecutive keys span at most δ: $k_{i+(\texttt{minPts}-1)} - k_i \le \delta$ for every valid i. We split the $m-1$ gaps into $\lceil (m-1)/(\texttt{minPts}-1) \rceil$ non-overlapping blocks of $\texttt{minPts}-1$ consecutive gaps; each block spans $\le \delta$ by the window constraint, so:

$$k_{m-1} - k_0 \;\le\; \left\lceil \frac{m-1}{\texttt{minPts}-1} \right\rceil \cdot \delta.$$

With $\delta = \texttt{minPts} \cdot \mathcal{L}/(2\varepsilon)$, the ceiling factor $\lceil (m-1)/(\texttt{minPts}-1) \rceil$ equals 1 for $m \le \texttt{minPts}$ (span $\le \delta < m\mathcal{L}/\varepsilon$, trivially satisfied) and first reaches 2 at $m = \texttt{minPts}+1$, where the margin is tightest:

$$2\delta \;=\; \texttt{minPts} \cdot \frac{\mathcal{L}}{\varepsilon} \;<\; (\texttt{minPts}+1) \cdot \frac{\mathcal{L}}{\varepsilon} \;=\; m \cdot \frac{\mathcal{L}}{\varepsilon}.$$

Hence every range from the forward sweep fits the exact-mode budget.

Merged segments. The merge step is conditional: two adjacent ranges are joined only if the combined span is still below $m'\mathcal{L}/\varepsilon$ for the combined size m'. This preserves the exact-mode invariant by construction.

False positives can therefore come only from the fallback filter, giving overall FPR $\le \varepsilon$.

5.3 Seg-ARE vs Grafite

Sections 5.2 and 5.2.4 have already told us *when* Seg-ARE is compact: when the input has dense regions separated by gaps the forward sweep can isolate. Below we work through two examples and count the bits.

Grafite, when the requested ε falls below the lossless threshold u/n, degenerates into a single Elias–Fano (EF) encoding of the raw sorted keys — the wrapper substitutes `ef_sux_vector(keys)` and the filter answers range queries by predecessor probe (Section 4.4.2). Below this threshold Grafite is an approximate filter; above it, it is essentially EF over the whole universe — the same exact/approximate switch as our Adaptive Filter (Section 4.3).

Seg-ARE applies the same EF primitive, but only after segmenting the input. Each dense cluster identified by the forward sweep is handed to an exact-mode ERE sub-filter (Section 4.3), which encodes the cluster's keys locally; sparse keys outside any cluster fall through to the truncation hash (Section 4.2). The total size of a Seg-ARE filter is the sum of per-cluster EF encodings plus the fallback truncation filter — versus Grafite's single EF encoding over the whole universe.

This framing makes the win condition concrete: Seg-ARE beats Grafite exactly when the per-cluster EF plus the fallback together cost less than one universe-wide EF. The bpk gap then decomposes as

$$\underbrace{\log_2(U_{\text{global}}/n)}_{\text{Grafite EF data}} \;-\; \underbrace{\Big(\sum_c \frac{n_c}{n} \cdot \log_2\!\big(2^{M_c}/n_c\big) \;+\; \frac{n_{\text{fb}}}{n} \cdot \log_2(1/\varepsilon)\Big)}_{\text{Seg-ARE: EF data + truncation fallback}},$$

where U_{global} is the spread of the whole input, 2^{M_c} and n_c are the local window width and key count of the c-th cluster, and n_{fb} is the number of keys handed to the fallback.

The two paragraphs below work through two examples: a synthetic clustered distribution and the SOSD-Wikipedia timestamps.

Clustered synthetic ($n = 2^{28}$). The clustered generator places keys across essentially the full 2^{64} universe (see Section 6.1), so $\log_2(U/n) = 36$ bits per key for the universe-wide baseline.

The forward sweep isolates 8911 dense clusters covering 72.9% of the keys, with $M_c \in [14.6, 28.7]$ bits and $n_c \in [512, 6.7\times10^7]$. The remaining 27.1% fall through to the truncation hash. The key-weighted per-cluster EF *lower-part* contribution is

$$\frac{1}{\sum_c n_c}\sum_c n_c \cdot \log_2\!\left(\frac{2^{M_c}}{n_c}\right) \;\approx\; 2.87 \text{ bits per in-cluster key}$$

— the remaining ≈ 7 bits per key come from metadata overhead and the truncation-fallback keys, giving ≈ 10 bits per key in total.

Grafite cannot trigger its lossless fallback (Section 4.4.2) at the requested ε either: lossless requires $n\mathcal{L}/\varepsilon > U$, i.e. $\varepsilon < n\mathcal{L}/U = 2^{28}\cdot 128/2^{64} \approx 1.9\times10^{-9}$. At any ε used in this evaluation Grafite stays in approximate mode and follows the Goswami bound $\log_2(\mathcal{L}/\varepsilon)+O(1)$. Universe-wide lossless EF on this input would saturate at ≈ 38.9 BPK — not a useful operating point for a filter whose whole purpose is to be much smaller than the keys themselves.

Figure 5.3 shows the empirical curve at query length $L = 128$. At matched FPR $\approx 6\times10^{-5}$, Seg-ARE sits at 9.77 BPK and Grafite at 20.63 BPK. Both filters then drop below the 4×10^{-6} measurement floor (set by the 10^6-query batch): Seg-ARE at 10.07 BPK and Grafite at 26.63 BPK; the true FPR at these points is non-zero but below the measurement floor, not exactly zero. The 16.5-BPK gap at the floor is therefore a lower bound on the BPK separation: matching FPRs deeper would widen the gap further, with Grafite's genuinely zero-FPR point only at the 38.94 BPK lossless saturation.

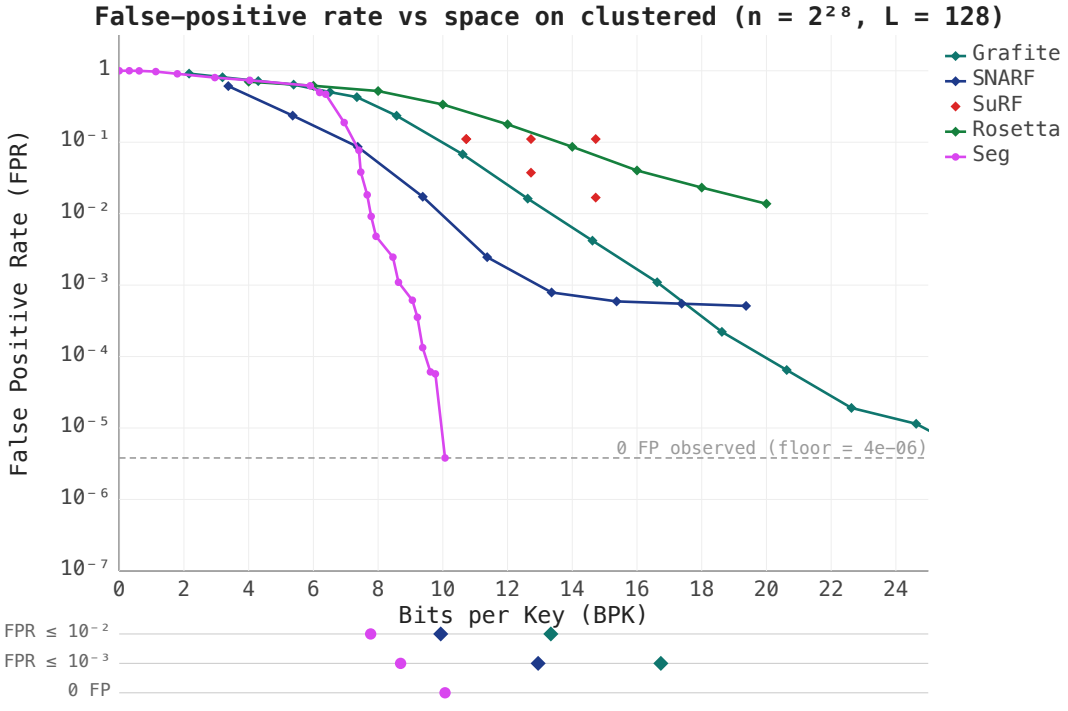

Figure 5.3: FPR vs BPK on the synthetic clustered distribution ($n = 2^{28}$, query length $L = 128$)

SOSD-Wikipedia ($n = 2^{28}$). Wikipedia edit timestamps form a tight near-sequential cluster: after deduplication $n \approx 9 \times 10^7$ keys span a window of width $\max - \min \approx 2.37 \times 10^8$ (about 29 bits, see Table 3.2). The segmenter does not collapse the input into a single cluster: the forward sweep finds 359 dense clusters covering 99.9% of the keys, with one giant cluster of 7.7×10^7 keys ($M_c = 26.5$ bits) and 358 smaller ones (M_c down to 14.6 bits). The full Seg-ARE filter uses 3.74 bits per key.

Grafite, on the same input, encodes the raw 29 bits universe. The EF coding saves $\approx \log_2(n)$ bits per key, that gives $29 - \log_2(9 \times 10^7) \approx 29 - 26.4 = 2.6$ bits per key. Together with the metadata overhead, it reaches FPR $= 0$ at 4.6 BPK in Figure 5.4. The 0.9-BPK gap to Seg-ARE is the normalization and segmentation saving — both filters use the same EF coding primitive.

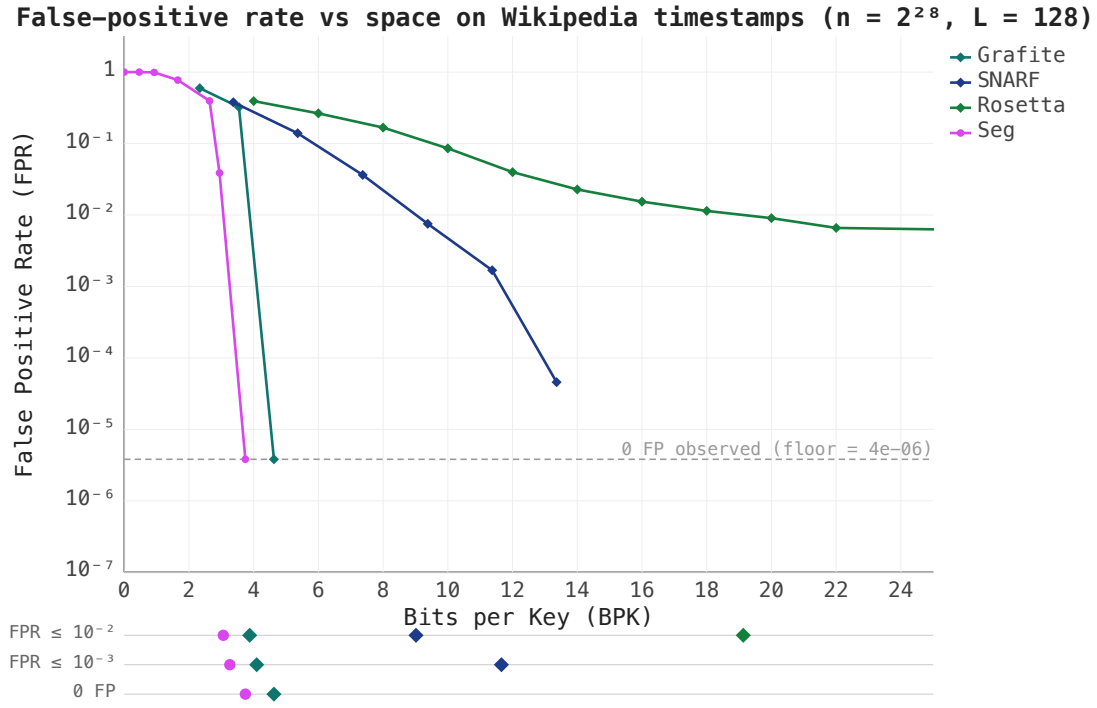

Figure 5.4: FPR vs BPK on SOSD-Wikipedia ($n = 2^{28}$, query length $L = 128$)

Uniform synthetic. Uniform is the case where Seg-ARE has no clusters to extract — every key goes to the truncation fallback. The truncation scheme (Section 4.2) pays $\log_2(1/\varepsilon) + O(1)$ bits per key *independently of* $\mathcal{L}$: large inter-key gaps make collisions rare, so the fixed-suffix length needed to hit a target ε does not have to grow with the query length. Grafite has no such mechanism and follows the Goswami bound $\log_2(\mathcal{L}/\varepsilon) + O(1)$, which grows with $\mathcal{L}$ by exactly $\log_2 \mathcal{L}$ bits. Table 6.6 (Uniform / gap-heavy column) shows the consequence at $\varepsilon = 10^{-3}$: at $\mathcal{L} = 1$ both filters sit at 12.4 BPK, at $\mathcal{L} = 16$ Seg-ARE is 15.1 BPK against Grafite's 16.6 BPK, and at $\mathcal{L} = 1024$ Grafite drops out of the measured range while Seg-ARE still reaches the target at 15.3 BPK. The widening gap is the missing $\log_2 \mathcal{L}$ factor.

Chapter 6

Experimental Evaluation

6.1 Benchmark Setup

Real-world keys come from the four SOSD datasets described in Section 3.1.2 (Facebook, Wiki, Books, OSM) and are loaded in their native bit-width. Synthetic distributions supplement them: uniform, spread, zipfian, and temporal generators mask their output to 60 bits, while the clustered generator emits full 64-bit keys (cluster centers and the uniform fraction are drawn from the whole `uint64` range). The only exception to native-width handling is SNARF: its upstream C++ implementation cannot accept keys close to `UINT64_MAX`, so the SNARF wrapper masks inputs to 60 bits internally. All other filters (Grafite, SuRF, Rosetta, and our ARE variants) operate on the full provided width.

Competing implementations:

- **Seg-ARE**: our simplified 1D DBSCAN hybrid filter (Section 5.2).
- **Grafite** [**?**]: information-theoretically optimal range filter (SIGMOD 2024).
- **SNARF** [**?**]: learned CDF-based range filter.
- **SuRF** [**?**]: succinct range filter with three variants (base, hash suffix, real suffix).
- **Rosetta** [**?**]: Bloom-hierarchy range filter (SIGMOD 2020).

All filters are evaluated on FPR-vs-BPK tradeoff curves, measured over 10^6 random empty-range queries per configuration — enough to register 10^3–10^4 false positives at our target FPR range of 10^{-2}–10^{-3} (Section 6.2, "FPR targets"). The query workload follows Section 3.2: FPR is measured on the empty-only stream (Section 3.2.1), latency on the combined stream (Section 3.2.2). The latency tables of Section 6.5 also report a small follow-up comparison between the two workloads.

Hardware. Unless stated otherwise, all measurements in this chapter were taken on an Apple M4 Max (macOS, 64 GB unified memory). The one exception is the ERE backend comparison of Section 6.3, which uses an x86-64 Linux machine (Linux 6.17, 62 GB RAM) because it relies on `perf_event_open` for hardware-counter measurements.

6.2 Workload Parameter Choices

Range filters in LSM-tree storage systems are built *per SSTable*: each on-disk file carries its own filter, and the relevant input size n is the number of keys in one SSTable, not the database total. Production deployments keep SSTables small — tens to hundreds of megabytes — for several reasons listed below.

1. **Memory and time, compounded.** The build of the filter during compaction operations needs random access to keys, so the working set — not just the filter — must be held in RAM during the operation. Filter construction itself runs at a few million keys per second: **?** report ∼7 M keys/s for Grafite at $n = 2 \times 10^8$ on a single thread. Both RAM and time scale linearly with n; compacting an $n = 2^{30}$ SSTable claims gigabytes of RAM and minutes of CPU on a single operation. This single-thread cost is unavoidable: in LevelDB, Cassandra, HBase, and Pebble each compaction job is single-threaded by design; RocksDB's optional subcompactions partition a job across threads but emit a separate SSTable per subcompaction, so the filter for one output file is still built by one thread [**?**].

2. **Cloud I/O.** Loading and saving SSTables on object storage backends (S3, GCS) is bandwidth-bound at the per-object level: a single connection to S3 sustains ∼55–95 MiB/s end-to-end [**?**], so a multi-gigabyte file streamed to a remote VM takes tens of seconds. Splitting an object across parallel uploads scales aggregate throughput further but competes with the live database for network bandwidth and CPU.

3. **Blast radius.** Corruption of one SSTable file or one object loses exactly the keys in that file. Smaller files bound the blast.

4. **Sharding.** Multiple smaller SSTables can be queried in parallel for the on-disk scans following a filter probe, migrated independently between nodes, and replicated at file granularity; one giant file cannot.

Realistic per-SSTable n. The default RocksDB `target_file_size_base` is 64 MB, with production deployments commonly tuning to 128 MB; entry sizes in real workloads (key + value) are roughly 60–150 bytes on average [**?**], putting the typical per-SSTable count at $n \in [2^{19},\ 2^{21}]$.

Two production cases push n higher. Secondary-index and counter column families (CFs — RocksDB's per-table partitioning, with independent in-memory write buffers (memtables) and SSTables) store all information in the key and use the value as a placeholder: in Facebook's social-graph workload, the `Assoc_count` CF has 20-byte bigint keys with values under 8 B, averaging ∼ 28 B/entry, and similar patterns appear in other index/counter CFs [**?**]. KV-separation deployments (WiscKey [**?**], RocksDB BlobDB, Pebble) store only a key plus a 10–20-byte blob handle in the SSTable, with values placed in separate blob files. In both cases the per-SSTable count rises to $n \in [2^{22},\ 2^{23}]$.

Range filter papers [**?????**] report headline numbers at $n \in [2^{24},\ 2^{28}]$, but this is a single-filter microbenchmark convention rather than a deployment claim: SuRF, SNARF, and Rosetta integrate per-SSTable into RocksDB; Memento targets B-Trees (WiredTiger, MongoDB's storage engine); Grafite is presented as a stand-alone structure. The SOSD benchmark [**?**] provides datasets at this same scale (200M keys $\approx 2^{27.6}$).

Why not a global filter. A database-wide filter is faster on the query side: range filter query time is essentially independent of n, so each per-file probe is constant-time — but a per-file design pays one such probe per SSTable, while a single global filter needs only one. The update side is the opposite story: LSM compaction atomically replaces SSTables, making per-file filter rebuild touch only the keys in that compaction, while a global filter must either rebuild over all keys or maintain counting structures that support deletion. GRF [**?**], a recent global range filter for LSM-trees, achieves single-probe queries through shape encoding; production systems — and our benchmarks — still adopt the simpler per-SSTable model.

The query-side argument is also weaker than it first looks. Within a level (L1+), RocksDB stores SSTables as a non-overlapping *sorted run*: each file carries [*smallest*, *largest*] key metadata in the MANIFEST (RocksDB's global level-metadata file), and a query does an $O(\log \#files)$ binary search on this metadata to identify only the SSTables whose key range overlaps the query [**?**], then probes their per-file filters [**?**].

Our choice of n. We benchmark at three anchor scales:

- $n = 2^{20}$ — per-SSTable production workload;
- $n = 2^{24}$ — large SSTable or aggregate filter;
- $n = 2^{28}$ — paper-evaluation and SOSD scale.

Per-key budget. Published range filter evaluations consistently report total per-key budgets in the 10–20 BPK range, with 14–16 BPK as the typical headline configuration (Table 6.1). The LSM point-filter convention sits at the lower end of this window: RocksDB ships Bloom filters at 10 BPK by default [**?**], reported by **?** as the cross-system standard. We adopt the same target range.

Filter	BPK budget	Reported FPR	Notes
SuRF [**?**]	10–16	$\sim 10^{-3}$ at 16 BPK*	RocksDB-anchored
Rosetta [**?**]	10–20	10^{-2} to 10^{-4}	FPR-vs-BPK sweep
SNARF [**?**]	16	$6.2 \times 10^{-5*}$	Headline result
Memento [**?**]	≥ 12	ε-parameterized	Design minimum
Grafite [**?**]	~ 12	$\leq \mathcal{L}/2^{\mathrm{BPK}-2}$	Adversarial bound

Table 6.1: Per-key memory budgets and corresponding false positive rates reported in recent range filter literature. FPR depends strongly on range length $\mathcal{L}$ and workload. *SuRF and SNARF FPR values at 16 BPK are both reported in **?** for direct comparison.

FPR targets. We report, for each filter, the BPK required to reach two fixed FPR targets: 10^{-2} and 10^{-3}. Both targets sit inside the operating range of published range filter evaluations: classical filters [**???**] anchor their headline numbers around 10^{-2} to 10^{-3}, while modern optimal filters [**???**] sweep deeper. They also align with the LSM point-filter convention: Cassandra's default `bloom_filter_fp_chance` is 0.01 for all compaction strategies except Leveled Compaction Strategy (LCS) [**?**].

Query range length $\mathcal{L}$. We sweep $\mathcal{L} \in \{1, 16, 128, 1024\}$. This range is production-aligned: in Facebook's UDB workload, more than 60% of iterator operations read exactly one key, the

P90 is below 100 keys, and scans above 10,000 are very rare [**?**]; the Yahoo! Cloud Serving Benchmark (YCSB) Workload E caps short-range scans at 100. Range filter papers [**????**] report headline FPR in this same window: SNARF at $\mathcal{L} = 256$, Rosetta at $\mathcal{L} = 64$, Grafite and Memento sweeping $\mathcal{L} \in \{1, 32, 1024\}$.

6.3 ERE Backend Comparison: Two-Vector vs One-Vector

We compare the two-vector and one-vector ERE backends as the exact layer of two ARE wrappers (SODA and Seg-ARE) across six distributions. The analytic prediction from Section 2.1.4 is $\Delta = M/n$ BPK, where M is the number of non-empty blocks; the maximum is $B/n = 1$ BPK and the Poisson middle for uniform input at $\lambda = 1$ is $1 - e^{-1} \approx 0.632$ BPK. The numbers below confirm the prediction end-to-end.

	SODA			Seg-ARE		
Dataset	two-vec BPK	one-vec BPK	Δ	two-vec BPK	one-vec BPK	Δ
`uniform`	16.932	16.499	0.433	16.932	16.499	0.433
`clustered`	16.602	16.500	0.102	12.759	12.519	0.240
`sosd_fb`	16.020	16.000	0.020	11.232	11.000	0.232
`sosd_wiki`	16.532	16.530	0.002	9.708	9.530	0.178
`sosd_osm`	16.931	16.499	0.432	26.678	26.411	0.267
`sosd_books`	16.000	16.000	0.000	4.517	4.000	0.517

Table 6.2: Total ARE-level BPK with the two-vector (D_1, D_2) ERE backend (Section 2.1.2) versus the one-vector D backend (Section 2.1.4). Workload: $n = 2^{20}$ (or all available keys, whichever is smaller), $K = 34$ — equivalent to $\mathcal{L} = 128$, $\varepsilon = 0.01$ via $K = \lceil \log_2(n\mathcal{L}/\varepsilon) \rceil$ for SODA; Seg-ARE takes K directly. SODA stays in the 14–16 BPK headline window (Table 6.1) on every distribution; Seg-ARE ranges from 4.0 BPK (`sosd_books`, single-cluster collapse) to 26.4 BPK (`sosd_osm`, many small clusters with per-cluster bookkeeping overhead).

SODA's locality-preserving hash keeps clustered inputs clustered in the hashed universe, so the SOSD datasets leave most blocks empty under SODA ($\Delta \leq 0.020$ BPK on `sosd_fb`, `sosd_wiki`, `sosd_books`); `uniform` and `sosd_osm` approach the analytic prediction (Section 2.1.5) at $\sim$0.43 BPK (because the benchmark evaluates at exactly $n = 2^{20}$, SODA hash collisions drop the unique fingerprint count slightly below the power-of-two threshold; this halves the ERE block count B to 2^{19}, shifting the load factor to $\lambda \approx 2$; the reported ARE BPK is logical, so it perfectly matches the analytic M/n prediction without physical succinct-index overheads). Seg-ARE isolates dense regions and builds a separate ERE per cluster; even within a cluster the keys retain sub-structure, so M/B stays well below 1. Its absolute BPK is dominated by the wrapper rather than ERE, so even small M/n values translate into a noticeable relative improvement (e.g. `sosd_books` drops from 4.517 to 4.000 BPK).

Query latency. Table 6.4 reports average ARE query latency on the same workload. Both wrappers gain uniformly across all distributions: SODA 1.05–3.59×, Seg-ARE 1.28–3.00×. The $\geq 2\times$ wins on `uniform` and `sosd_osm` are instruction-driven: SODA's 2-universal hashing maps both to a near-uniform image in $[0, 2^K)$, so the merged D vector's SELECT chains are short. The smaller speedups on `clustered`, `sosd_fb`, and `sosd_wiki` match their 34–39% instruction reduction; `sosd_books` gains least because its power-law distribution leaves many hashed buckets empty even after hashing, lengthening SELECT chains and limiting the instruction saving

to 12%. At $n = 2^{20}$, cache miss counts stay below one per query—the filter fits in the L3 cache—so cache effects are secondary.

Dataset	Δ instrs	Δ L1-loads	Δ L1-misses
`uniform`	−76%	−81%	−56%
`clustered`	−34%	−35%	−20%
`sosd_fb`	−39%	−39%	−21%
`sosd_wiki`	−39%	−38%	−14%
`sosd_osm`	−76%	−81%	−56%
`sosd_books`	−12%	−13%	4%

Table 6.3: Per-query reduction in instructions executed, L1 cache loads, and L1 cache misses when switching from the two-vector to the one-vector ERE backend inside SODA ARE. Measured with `perf_event_open` (user-space only) on an x86-64 Linux machine (Linux 6.17, 62 GB RAM), pinned to a single core; $n = 2^{20}$, $K = 34$, range length 128, 100 000 queries (10 000 warmup).

	SODA			Seg-ARE		
Dataset	two-vec ns	one-vec ns	speedup	two-vec ns	one-vec ns	speedup
`uniform`	324.1	90.6	3.58×	218.6	72.9	3.00×
`clustered`	381.8	287.5	1.33×	317.9	198.3	1.60×
`sosd_fb`	445.6	309.5	1.44×	224.9	158.0	1.42×
`sosd_wiki`	424.7	327.3	1.30×	303.3	206.5	1.47×
`sosd_osm`	334.9	93.4	3.59×	298.3	232.6	1.28×
`sosd_books`	323.7	308.4	1.05×	223.1	104.7	2.13×

Table 6.4: Average ARE query latency, two-vector vs one-vector ERE backend. Same workload as Table 6.2: $n = 2^{20}$ (or all available keys, whichever is smaller), $K = 34$; raw 64-bit keys (no 60-bit mask). x86-64 Linux machine (Linux 6.17, 62 GB RAM), pinned to a single core; 100 000 queries (10 000 warmup). This is a small-scale backend comparison at fixed $n = 2^{20}$; absolute latencies grow 5–10× at the headline scale $n = 2^{24}$ once the filter footprint outgrows the L3 cache and per-query work becomes cache-miss-bound.

6.4 ERE Bucket Occupancy on SOSD

To characterize how the SODA hash distributes input keys across ERE blocks on real workloads, we measured per-bucket statistics for each SOSD dataset across three range lengths.

Dataset	n	$\mathcal{L}$	w	X_{50}	X_{90}	Max	Max footprint	Fill
`sosd_fb`	2^{27}	16	11	27	103	463	637 B	22.6%
`sosd_fb`	2^{27}	256	15	208	545	1,589	2.9 KB	4.9%
`sosd_wiki`	$\sim 2^{26*}$	16	12	3,240	3,843	4,054	5.9 KB	**99.0**%
`sosd_wiki`	$\sim 2^{26*}$	256	16	52,565	58,855	62,037	121 KB	**94.7**%
`sosd_books`	2^{27}	16	11	812	875	978	1.3 KB	47.8%
`sosd_books`	2^{27}	256	15	13,215	13,670	14,039	25.7 KB	42.8%
`sosd_osm`	2^{29}	16	11	3	5	35	48 B	1.7%
`sosd_osm`	2^{29}	256	15	3	5	85	159 B	0.3%

Table 6.5: Bucket-size statistics for the inner ERE of the SODA-hashed ARE filter on the SOSD datasets across two range lengths covering the typical Facebook UDB range ($\mathcal{L} = 16$) and its P90 ($\mathcal{L} = 256$). *`sosd_wiki` effective $n \approx 9 \times 10^7$ after deduplication of repeated timestamps (Section 3.1.2); reported as $\sim 2^{26}$ for visual consistency with the other rows.

Columns.

w — Suffix length in bits, configured per row. Physical block capacity is 2^w distinct w-bit suffixes.

X_{50}, X_{90} — Key-weighted CDF percentiles over non-empty buckets: the smallest bucket size such that buckets of size $\leq X_p$ collectively hold at least $p \cdot n$ keys.

Max — Empirical maximum bucket size, $\max_b k_b$.

Max footprint — Bit-packed size of the heaviest bucket ($\max_b k_b \cdot w/8$ bytes).

Fill — $\max_b k_b / 2^w$ — the heaviest bucket's saturation against physical capacity.

Observations. `sosd_fb` and `sosd_osm` stay well below saturation at every $\mathcal{L}$. `sosd_books` runs at ∼42% across $\mathcal{L}$. `sosd_wiki` fully saturates at $\mathcal{L} = 16$ (99%). In all cases except `sosd_wiki` at $\mathcal{L} = 256$, bucket footprints fit L1d on modern x86 servers, typically 32–48 KB. The saturated wiki bucket (121 KB) fits L2.

When fill approaches 100%, the SODA hash has run out of resolution: every key in a dense input region collapses into a single ARE block, and the bucket-search cost for those queries becomes $O(2^w)$ rather than $O(1)$ on average. For `sosd_wiki` this means binary search on a fully packed bucket of up to $\sim 6 \times 10^4$ keys (121 KB at $\mathcal{L} = 256$) — still bounded by w iterations and cache-resident in L2 (Section 2.1.5), but the buckets are no longer lightly loaded as the SODA construction assumes.

6.5 Comparison Tables

Each table below reports results at the configuration that achieves FPR $\leq 10^{-3}$ with the fewest bits per key ($\varepsilon = 10^{-3}$, gap-heavy query workload, $n = \min(2^{28}, |\text{dataset}|)$). A dash (—) means the filter did not reach the target FPR within the measured parameter range. **Bold** marks the best result in each column (lowest BPK / lowest latency / highest throughput), excluding dashes.

Reported BPK. BPK is the smallest measured value at which FPR $\leq 10^{-3}$. Latency and throughput are taken from that same point.

6.5.1 Bits per Key to Reach FPR $\leq 10^{-3}$

	Smart-mix (correlated, 100% empty)						Gap-heavy (100% empty)					
	SOSD				Synthetic		SOSD				Synthetic	
Filter	FB	OSM	Wiki	Books	Unif	Clust	FB	OSM	Wiki	Books	Unif	Clust
Point queries, $\mathcal{L} = 1$												
SuRF	17.7	—	—	—	—	—	16.9	20.9	—	17.6	17.3	32.4
SNARF	12.4	—	8.8	—	—	—	13.4	—	8.7	—	15.1	12.5
Rosetta	15.5	14.8	15.8	14.9	14.8	14.9	15.8	14.7	16.0	14.7	14.9	14.8
Grafite	**10.8**	**12.6**	4.7	**12.9**	12.6	14.7	**11.3**	12.5	4.6	12.6	**12.4**	11.9
Seg-ARE	12.6	13.5	**3.7**	13.5	**12.5**	**7.7**	**11.3**	**12.5**	**3.7**	**12.4**	**12.4**	**7.8**
Short queries, $\mathcal{L} = 16$												
SuRF	17.5	—	—	—	—	—	16.8	20.9	—	17.6	17.3	31.8
SNARF	11.7	—	9.7	—	—	—	12.0	—	10.2	—	**15.0**	12.6
Rosetta	20.5	20.1	20.1	20.2	20.1	19.4	20.3	20.0	20.3	20.3	20.0	19.3
Grafite	**11.3**	16.6	4.9	16.6	16.6	14.6	**11.3**	16.6	4.6	16.8	16.6	14.6
Seg-ARE	11.4	**16.5**	**3.7**	**16.5**	**16.5**	**7.9**	**11.3**	**16.3**	**3.7**	**16.7**	15.1	**7.9**
Long queries, $\mathcal{L} = 1024$												
SuRF	18.1	—	—	—	—	—	18.0	**24.2**	—	**17.7**	17.2	34.9
SNARF	11.9	—	12.7	—	—	—	12.4	—	12.3	—	15.4	13.6
Rosetta	46.2	41.8	47.2	41.1	41.3	43.5	44.8	42.1	47.4	42.8	41.7	43.1
Grafite	**11.3**	22.9	4.6	22.6	22.6	18.9	**11.3**	22.5	4.6	22.4	22.7	18.9
Seg-ARE	11.4	**22.5**	**3.7**	**22.5**	**22.5**	**9.4**	**11.3**	22.5	**3.7**	22.3	**15.3**	**9.3**

Table 6.6: BPK to reach FPR $\leq 10^{-3}$ for point ($\mathcal{L} = 1$), short ($\mathcal{L} = 16$), and long ($\mathcal{L} = 1024$) queries. $n = \min(2^{28}, |\text{dataset}|)$. Left half: smart-mix workload (50% near-key, 30% in-gap, 20% uniform, all truncated to be empty). Right half: gap-heavy workload (0% near-key). Both columns measure FPR on 100% empty queries; smart-mix is the correlated workload where SuRF/Rosetta show their worst-case behavior, gap-heavy is the benign baseline. Dash: target FPR not reached within the measured BPK range.

Across flat distributions (FB, OSM, Books, Uniform) Seg-ARE matches Grafite to within 0.5 BPK. On structured distributions — Wikipedia edit timestamps and the synthetic clustered set — Seg-ARE saves 1–10 BPK.

6.5.2 Query Latency at the Chosen Configuration

Query latency (ns/op) measured at the same configuration as the BPK table above (minimum BPK that achieves FPR $\leq 10^{-3}$). An asterisk (*) marks cells where the filter did not reach the target FPR within the measured BPK range; the value is the latency at the lowest achieved FPR under the smart-mix workload, shown as a query-time floor rather than the latency at the chosen configuration.

	Smart-mix empty (with truncation)						Smart-mix mixed (no truncation)					
	SOSD				Synthetic		SOSD				Synthetic	
Filter	FB	OSM	Wiki	Books	Unif	Clust	FB	OSM	Wiki	Books	Unif	Clust
Point queries, $\mathcal{L} = 1$												
SuRF	427	677*	159*	588*	604*	643*	524	677*	159*	588*	604*	643*
SNARF	982	974*	693	953*	1017*	1555*	980	974*	651	953*	1017*	1555*
Rosetta	**137**	**181**	**134**	**162**	**157**	**160**	**169**	**186**	**183**	**177**	**188**	**188**
Grafite	328	348	278	299	299	493	385	388	277	347	378	542
Seg-ARE	404	192	146	197	191	321	410	224	199	216	218	333
Short queries, $\mathcal{L} = 16$												
SuRF	410	597*	153*	712*	515*	748*	506	597*	153*	712*	515*	748*
SNARF	1005	1043*	692	758*	1214*	926*	928	1043*	575	758*	1214*	926*
Rosetta	1797	2531	676	2418	2462	1551	1843	3511	882	2729	2447	1445
Grafite	**322**	300	266	305	307	499	408	396	290	357	318	361
Seg-ARE	373	**227**	**170**	**197**	**192**	**326**	**407**	**250**	**232**	**231**	**224**	**214**
Long queries, $\mathcal{L} = 1024$												
SuRF	383	649*	201*	526*	530*	616*	470	649*	201*	526*	530*	616*
SNARF	987	1129*	689	746*	1286*	1041*	830	1129*	554	746*	1286*	1041*
Rosetta	4305	16767	816	16064	14602	4584	4678	20318	1339	15352	18324	4238
Grafite	**307**	355	219	334	1211	644	**348**	332	292	347	366	492
Seg-ARE	394	**231**	**169**	**199**	**195**	**335**	421	**236**	**225**	**229**	**224**	**215**

Table 6.7: Query latency (ns/op) at the same configuration as Table 6.6, for point ($\mathcal{L} = 1$), short ($\mathcal{L} = 16$), and long ($\mathcal{L} = 1024$) queries. Left half: smart-mix workload with truncation (Section 3.2.1), so every query is empty and the filter exercises only its "no match" code path. Right half: smart-mix workload without truncation (Section 3.2.2), so a fraction of the queries actually contain stored keys. Hardware: Apple M4 Max, single core, 10^6 queries per cell (mean ns/op).

Empty-only vs combined queries. To check that the ordering between filters survives a realistic mix of empty and non-empty queries, we re-ran the same configurations with the combined workload of Section 3.2.2 (no truncation), at $n = 2^{24}$ on the four SOSD datasets and $L \in \{1, 16, 1024\}$. The ordering between filters is preserved: at $\mathcal{L} = 1$ Rosetta is fastest (Table 6.7, $\mathcal{L} = 1$ block) and Seg-ARE a close second; at $\mathcal{L} \geq 16$ Seg-ARE is fastest and Rosetta degrades steeply (Table 6.7, $\mathcal{L} \geq 16$ blocks), while SNARF stays among the slowest at every $\mathcal{L}$. We report the empty-only numbers as headline figures because they let FPR and latency be measured on the same query batch.

Why Rosetta is fastest at $\mathcal{L} = 1$. At $\mathcal{L} = 1$ Rosetta's Bloom-cascade query (Section 1.2, paragraph Bloom cascade: Rosetta) degenerates to a single probe of the deepest Bloom level. The per-level probe cost that hurts it at $\mathcal{L} \geq 16$ is not paid here.

6.5.3 Build Throughput at Operating Point

Build throughput (M keys/s) measured at the same configuration as the BPK table above. An asterisk (*) marks cells where the filter did not reach the target FPR within the measured BPK range; the value is the throughput at the highest measured BPK (same proxy point as the asterisked latency entries).

	SOSD				Synthetic	
Filter	FB	OSM	Wiki	Books	Uniform	Clustered
Point queries, $\mathcal{L} = 1$						
SuRF	26.8	**26.8**	36.8*	**27.8**	**23.8**	28.1
SNARF	11.0	8.1*	14.5	8.2*	10.1	9.5
Rosetta	3.8	2.8	3.8	2.7	2.8	2.8
Grafite	**54.5**	23.3	34.9	23.6	23.5	35.7
Seg-ARE	41.3	25.6	**57.1**	24.8	23.2	**38.6**
Short queries, $\mathcal{L} = 16$						
SuRF	26.8	**26.8**	36.9*	**27.8**	23.8	28.1
SNARF	11.0	8.0*	13.3	8.1*	10.1	9.5
Rosetta	2.1	2.1	3.0	2.3	2.2	2.3
Grafite	**54.5**	22.8	34.9	22.9	23.1	36.0
Seg-ARE	41.3	22.3	**57.1**	25.1	**73.0**	**44.1**
Long queries, $\mathcal{L} = 1024$						
SuRF	26.8	**26.8**	36.6*	**27.8**	23.8	28.1
SNARF	11.0	8.0*	11.7	8.2*	10.1	9.1
Rosetta	1.1	1.2	1.3	1.2	1.3	1.2
Grafite	**54.5**	22.4	34.9	23.1	21.6	38.4
Seg-ARE	41.3	23.4	**57.1**	25.1	**73.0**	**44.1**

Table 6.8: Build throughput (M keys/s) at the same configuration as Table 6.6, for point ($\mathcal{L} = 1$), short ($\mathcal{L} = 16$), and long ($\mathcal{L} = 1024$) queries. Hardware: Apple M4 Max, single core.

Chapter 7

Conclusion

7.1 Summary of Contributions

- **Single-vector ERE backend** (Chapter 2): consolidates two-layer metadata into one bit-vector, cutting metadata size by 24%.
- **Adaptive bucket-search** (Chapter 2): drops 59 bits of per-key metadata and speeds queries by 13–30× at the bucket sizes that arise at per-SSTable scale.
- **Truncation hash** (Chapter 4): saves $\log_2 \mathcal{L}$ bits per key on sparse inputs, going below the Goswami bound on distributions where keys are spread across the universe.
- **Seg-ARE** (Chapter 5): a single-pass segmentation that routes dense clusters to exact-mode ERE sub-filters and sparse tails to the truncation hash, going below the bound on structured distributions.

The first two improvements bring the Goswami-line filter closer to the lower bound on adversarial inputs; the truncation hash and Seg-ARE each cross below it on inputs where the respective structure is present.

In absolute terms, the combined filter achieves the following on standard SOSD benchmarks at FPR $\leq 10^{-3}$:

- **3.7 bits per key** on Wikipedia timestamps — less than a third of any competitor at the same error rate.
- **7.7–9.2 bits per key** on clustered synthetic keys, versus 12–19 for the next-best filter.
- **150–420 ns** per query (Apple M4 Max, single core) across all four SOSD real-world datasets and query lengths $\mathcal{L} \in \{1, 16, 1024\}$ (Table 6.7).

7.2 When Seg-ARE Wins, Ties, and Loses

The empirical picture across the four SOSD distributions falls out of the per-cluster vs universe-wide Elias–Fano decomposition of Section 5.3:

- **Wiki — clearest win.** Wikipedia edit timestamps are heavily concentrated on the right side of the universe (Figure 3.1). The segmentation isolates the dense parts cleanly;

per-cluster EF encodes each tight region in far fewer bits than a universe-wide EF could. Seg-ARE reaches FPR = 0 at 3.7 BPK against Grafite's 4.6 BPK at the lossless threshold (Figure 5.4, Table 6.6, $\mathcal{L} = 16$ block).

- **Facebook — tied.** Facebook user IDs occupy a relatively small 37-bit universe with no large gaps. There is little for segmentation to exploit, and universe-wide EF is already compact. Seg-ARE and Grafite both reach FPR = 0 at ~ 11 BPK; Seg-ARE matches but does not improve over the Grafite-with-fallback baseline.
- **OSM — no advantage.** OpenStreetMap cell IDs are spread thinly across the full 64-bit universe, with median gaps of order 10^7. The DBSCAN-style detector finds essentially no clusters above the $\delta = 256\mathcal{L}/\varepsilon$ window, so most keys go to the truncation-hash fallback and Seg-ARE degrades to truncation. Honest hashing is the only option on this geometry; no segmentation primitive helps.

Seg-ARE is therefore not a universal improvement over Grafite. It is a distribution-aware option that activates when the data has the cluster-and-gap geometry common in many real LSM key sets (Wiki-like clustered timestamps) and matches Grafite on the rest.

7.3 Limitations

The Seg-ARE design carries three honest limitations.

Detector sensitivity. The forward sweep depends on $\delta = \texttt{minPts} \cdot \mathcal{L}/(2\varepsilon)$ and on the fixed choice `minPts` = 512 (Chapter 5). On clusters smaller than `minPts` keys the detector misses real density and the win is lost. We have not investigated adaptive `minPts` tuning.

Per-segment overhead. Each segment carries ~ 700 bits of fixed metadata (cluster bounds, sub-filter header, pointer). At `minPts` = 512 this is ~ 1.4 BPK of fixed overhead. On inputs that produce many small clusters the overhead can outweigh the per-cluster EF savings.

Static setting. The construction takes a fixed key set; segments are not maintained under inserts or deletes. Dynamic key sets, where the cluster structure itself evolves, require the keepsake-box scheme of Memento [**?**] or an equivalent online segmentation mechanism.

7.4 Future Work

Three directions stand out.

Adaptive segmentation parameters. The window and density thresholds are currently fixed at construction time from a single `minPts` value. Choosing them per LSM level (top levels have fewer keys but tighter clusters; bottom levels are denser but flatter) could close the gap on OSM-like distributions where the global δ is too coarse.

Combining segmentation with learned models. The CDF-fitting approach of SNARF [**?**] solves the universe-mapping problem that hurts Grafite on small-universe inputs, but loses the worst-case guarantee. Running a segmenter first, then a CDF model on each cluster, would localise the model's assumptions and might recover the guarantee on the clusters individually.

Dynamic Seg-ARE. Combining the segmentation primitive with Memento's dynamic keepsake-box machinery would extend the distribution-aware win condition from static SSTa-

bles to mutable indexes (B-trees, single-global-filter LSM-trees, mixed transactional/analytical workloads).